

Bazele Inteligenței Artificiale

Lab 6. Clusterizare

Efrem Dragoș-Sebastian-Mihaly

Tipuri de modele în ML

- În general, sunt două tipuri mari și late:

Supervizate

Nesupervizate

Supervizate

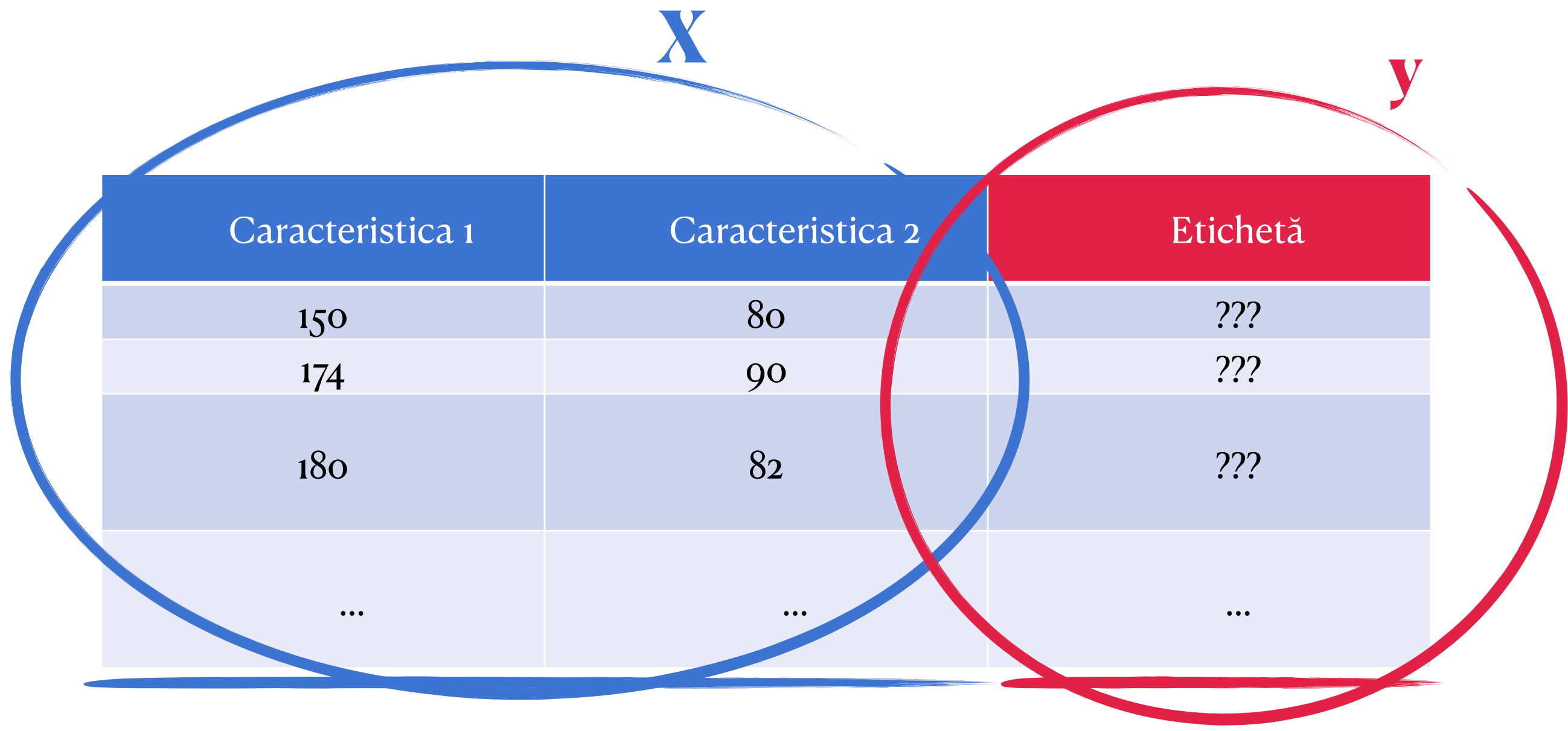
- Învată din date care au deja etichete (adică știm răspunsul corect pentru fiecare element utilizat pentru a antrena modelul)



Nesupervizate

- Învăță din date care nu au etichete (adică nu știm răspunsul corect pentru fiecare element utilizat pentru a antrena modelul)
- Spre deosebire de învățarea supervizată, unde îi spunem algoritmului exact ce reprezintă fiecare categorie (ex: “aceasta e o pisică, acesta e un câine”), în învățarea nesupervizată îi dăm pur și simplu o mulțime de date și îl lăsăm să identifice singur asemănările sau grupările
- Imaginați-vă că intrați într-o cameră plină de oameni fără să aveți habar cine sunt
- Dacă observați comportamentul lor și cine cu cine interacționează, veți putea intui grupuri sau categorii. Cam așa funcționează învățarea nesupervizată: descoperirea de tipare fără să știm de la început detalii precise

Să zicem că avem un set de date în 2D



Caracteristici / Atribute / Features

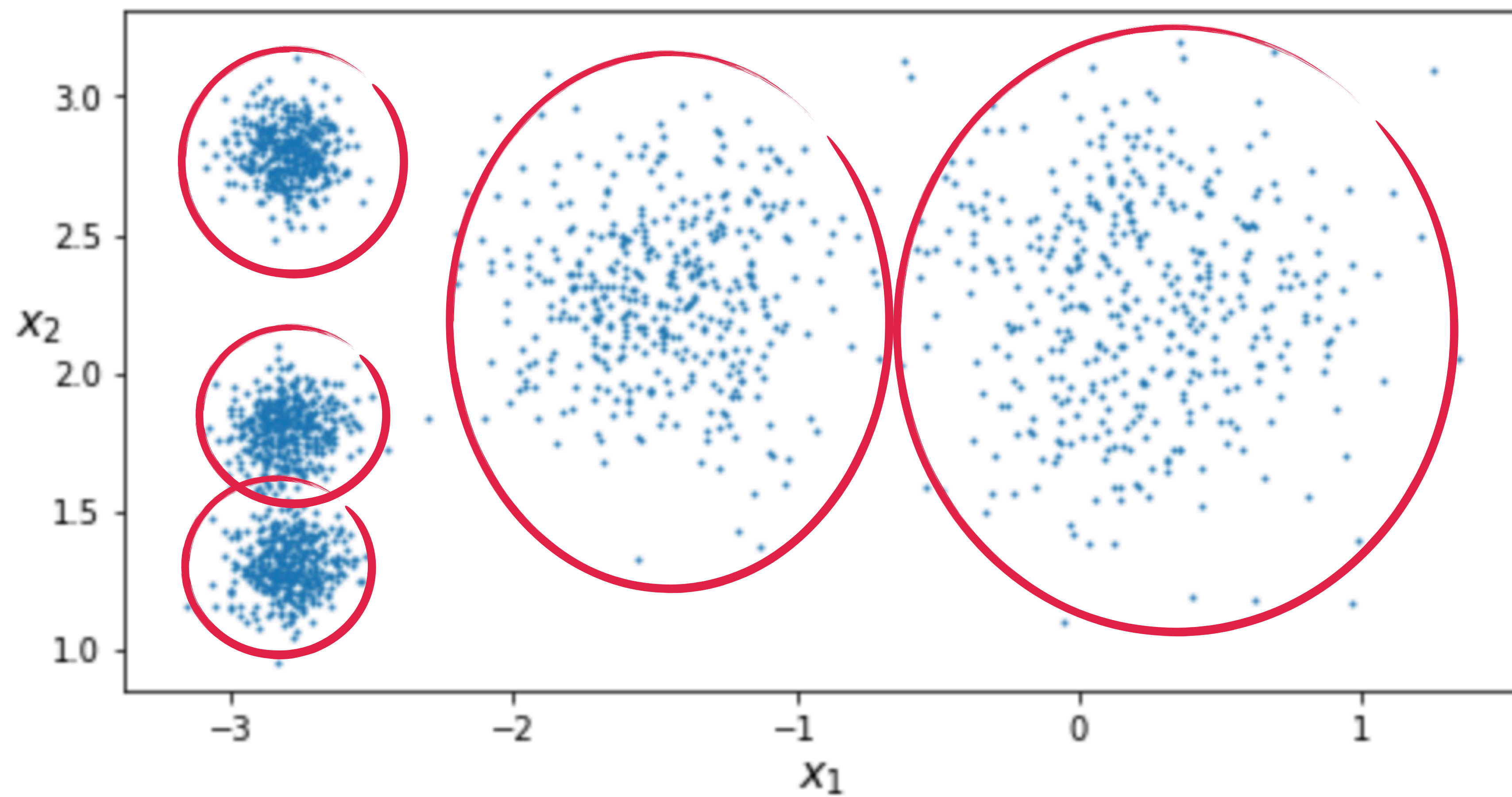
Etichetă / Clasă

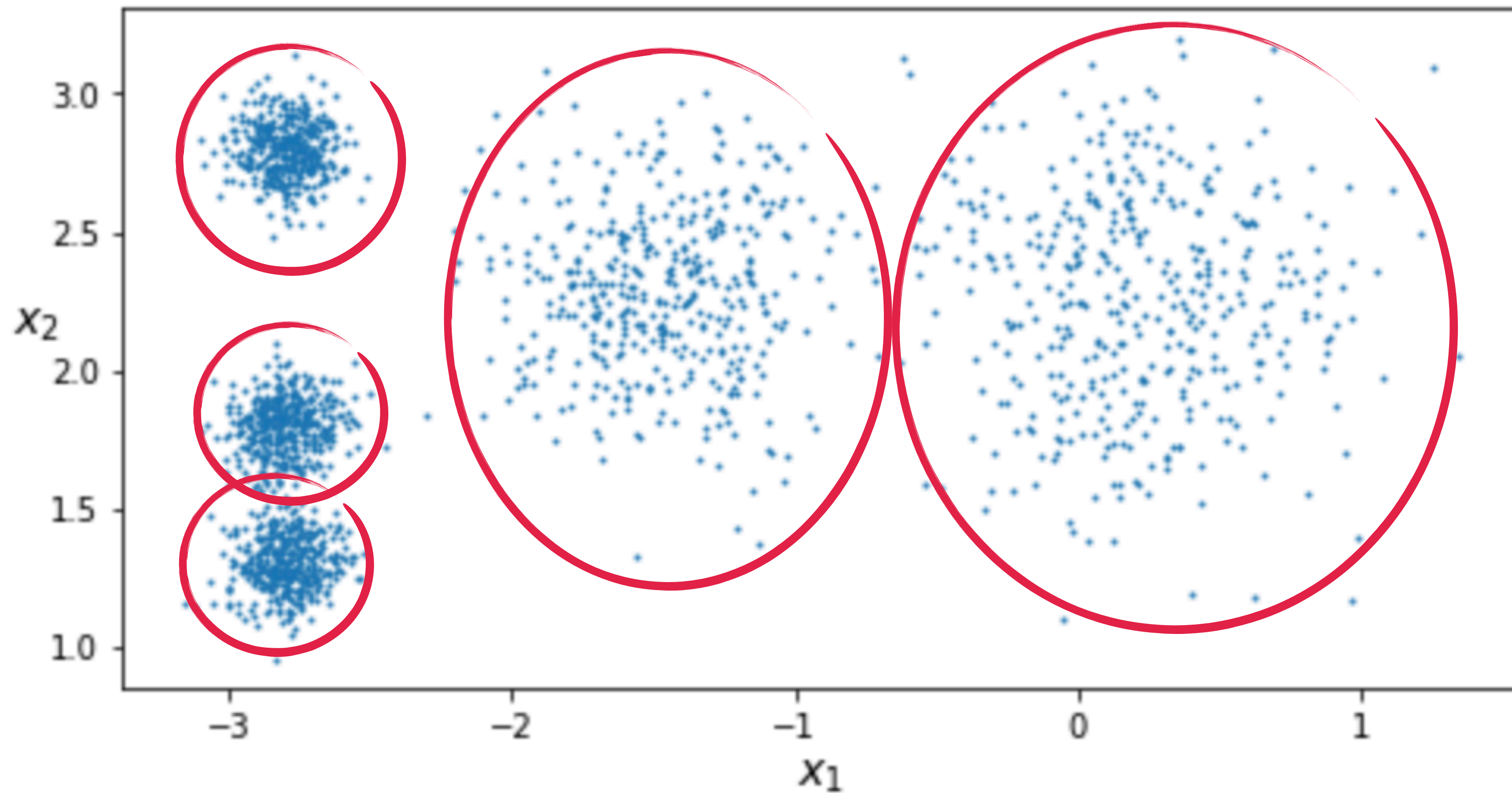
Să zicem că avem un set de date în 2D

X

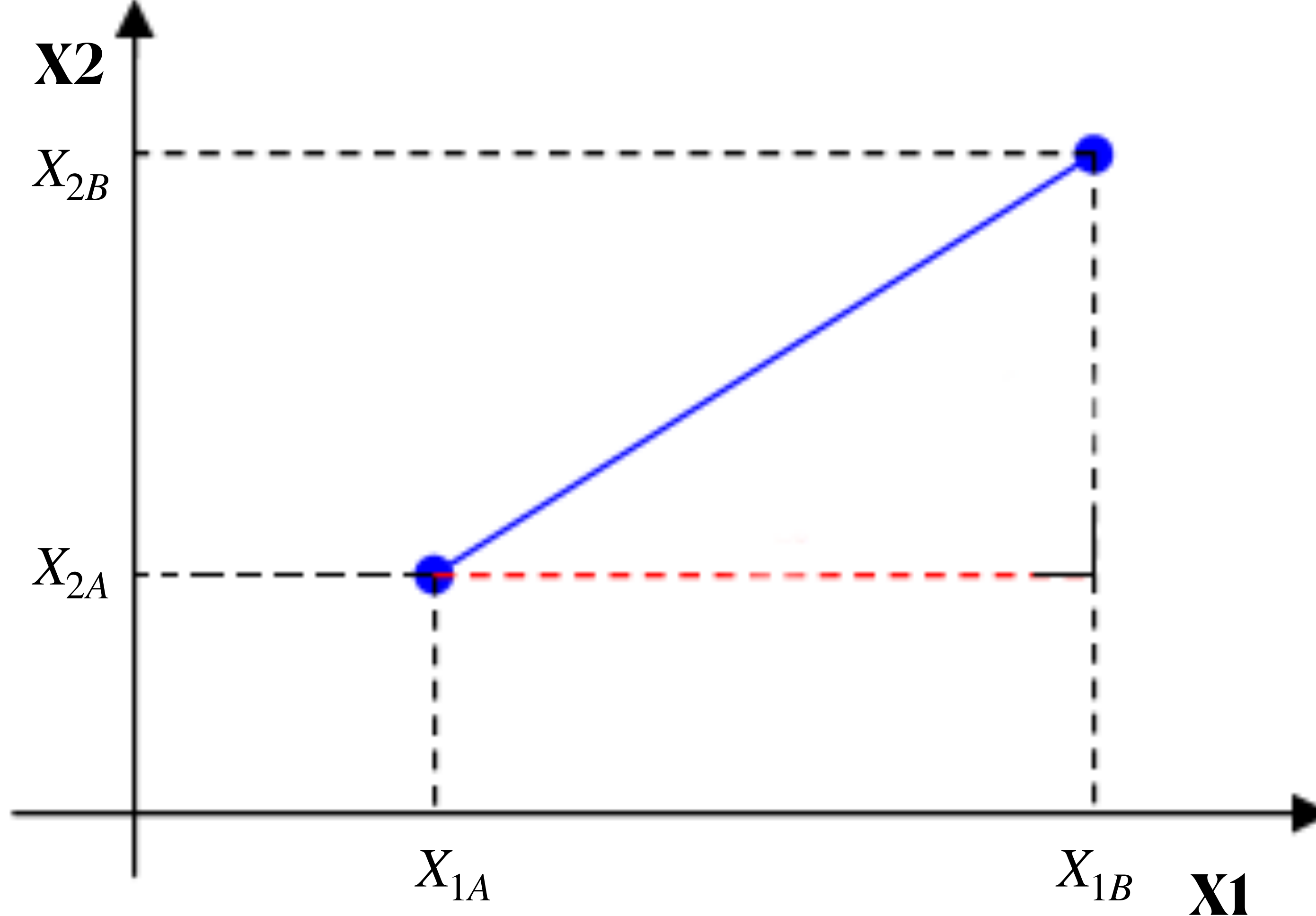
Caracteristica 1	Caracteristica 2
150	80
174	90
180	82
...	...

Caracteristici / Attribute / Features

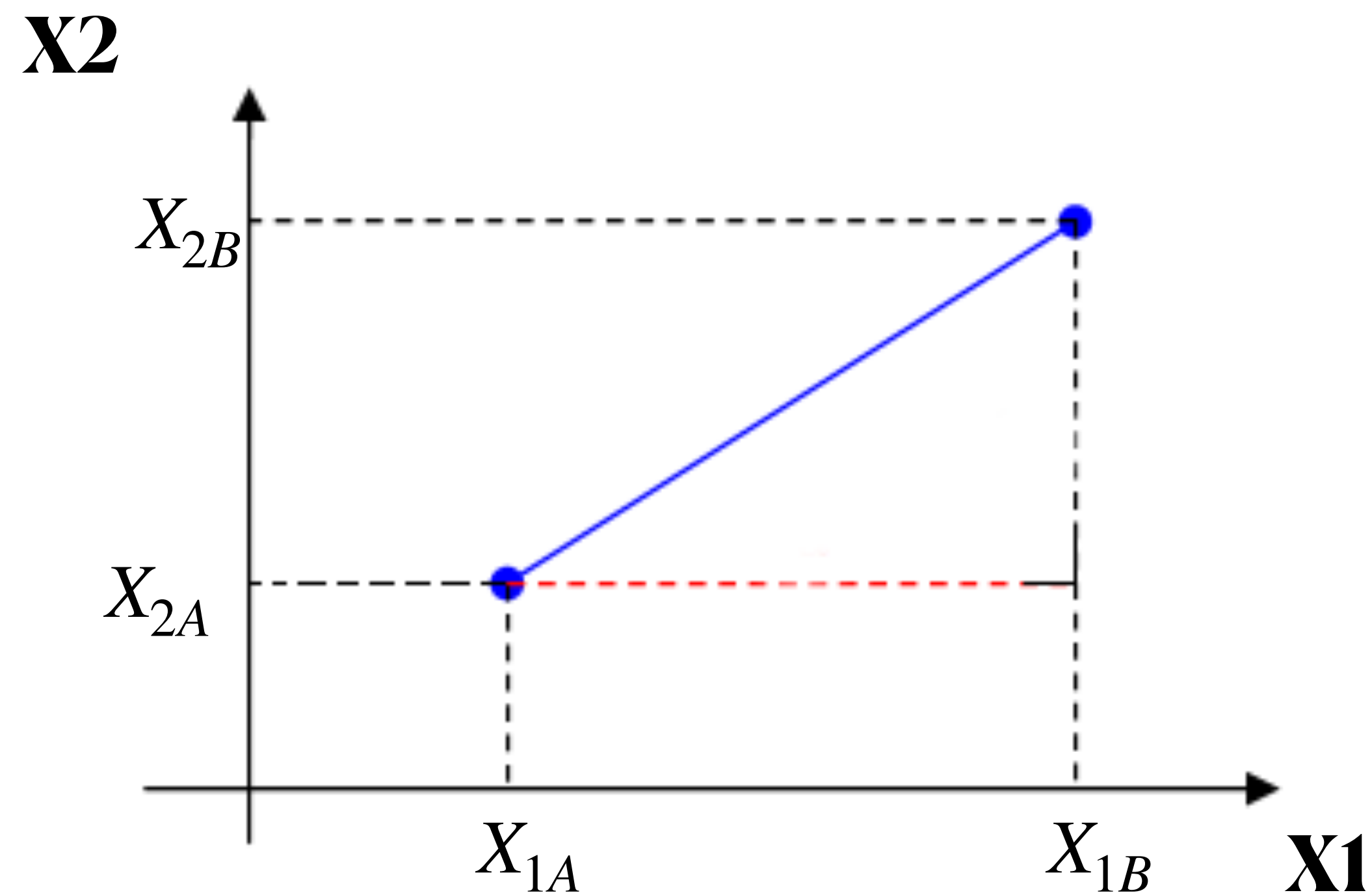




- Ochiometric, oamenii își pot da seama ușor de grupuri în date
- **Problema** e cum facem un model care să își dea seama singur de aceste grupuri
- Din poză, observăm că distanța dintre puncte poate fi folosită ca o măsură a similarității



- Avem două puncte de date, (X_{1A}, X_{2A}) și (X_{1B}, X_{2B})
- Distanța dintre ele poate fi calculată ca distanța euclidiană: $d = \sqrt{(X_{1B} - X_{1A})^2 + (X_{2B} - X_{2A})^2}$
- Observăm că dacă $X_{1A} = X_{1B}$ și $X_{2A} = X_{2B}$, cele două puncte sunt de fapt același punct și atunci distanța e zero



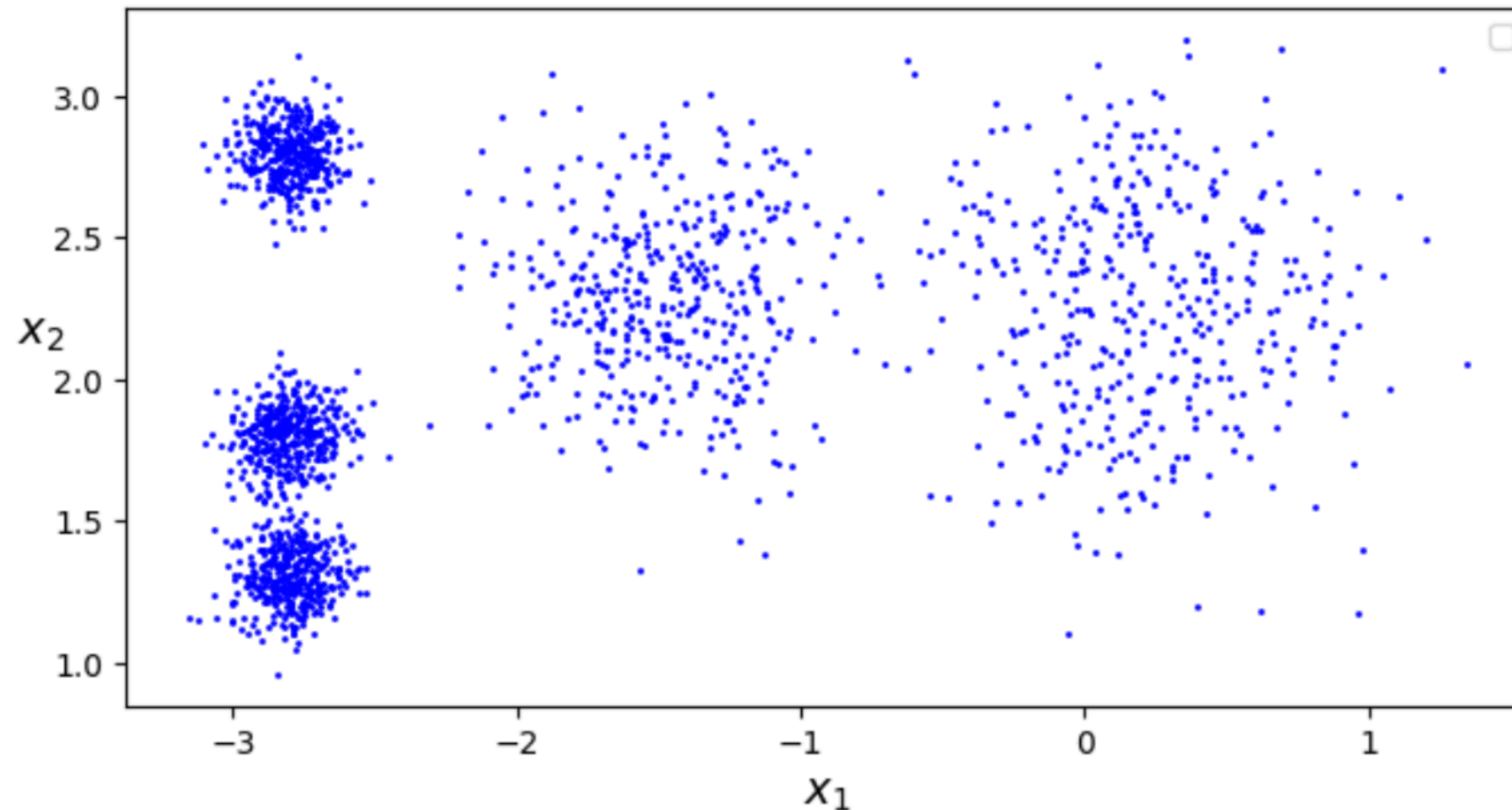
- Avem două puncte de date, (X_{1A}, X_{2A}) și (X_{1B}, X_{2B})
- Distanța dintre ele poate fi calculată ca distanța euclideană: $d = \sqrt{(X_{1B} - X_{1A})^2 + (X_{2B} - X_{2A})^2}$
- Observăm că dacă $X_{1A} = X_{1B}$ și $X_{2A} = X_{2B}$, cele două puncte sunt de fapt același punct și atunci distanța e zero
- Are sens: dacă punctele prezintă exact aceleași caracteristici, atunci sunt maxim de similare
- Cu cât distanța dintre două puncte este mai aproape de zero, cu atât ele sunt mai similare din punctul de vedere al caracteristicilor

Clusterizarea K-Means

Clusterizarea K-Means

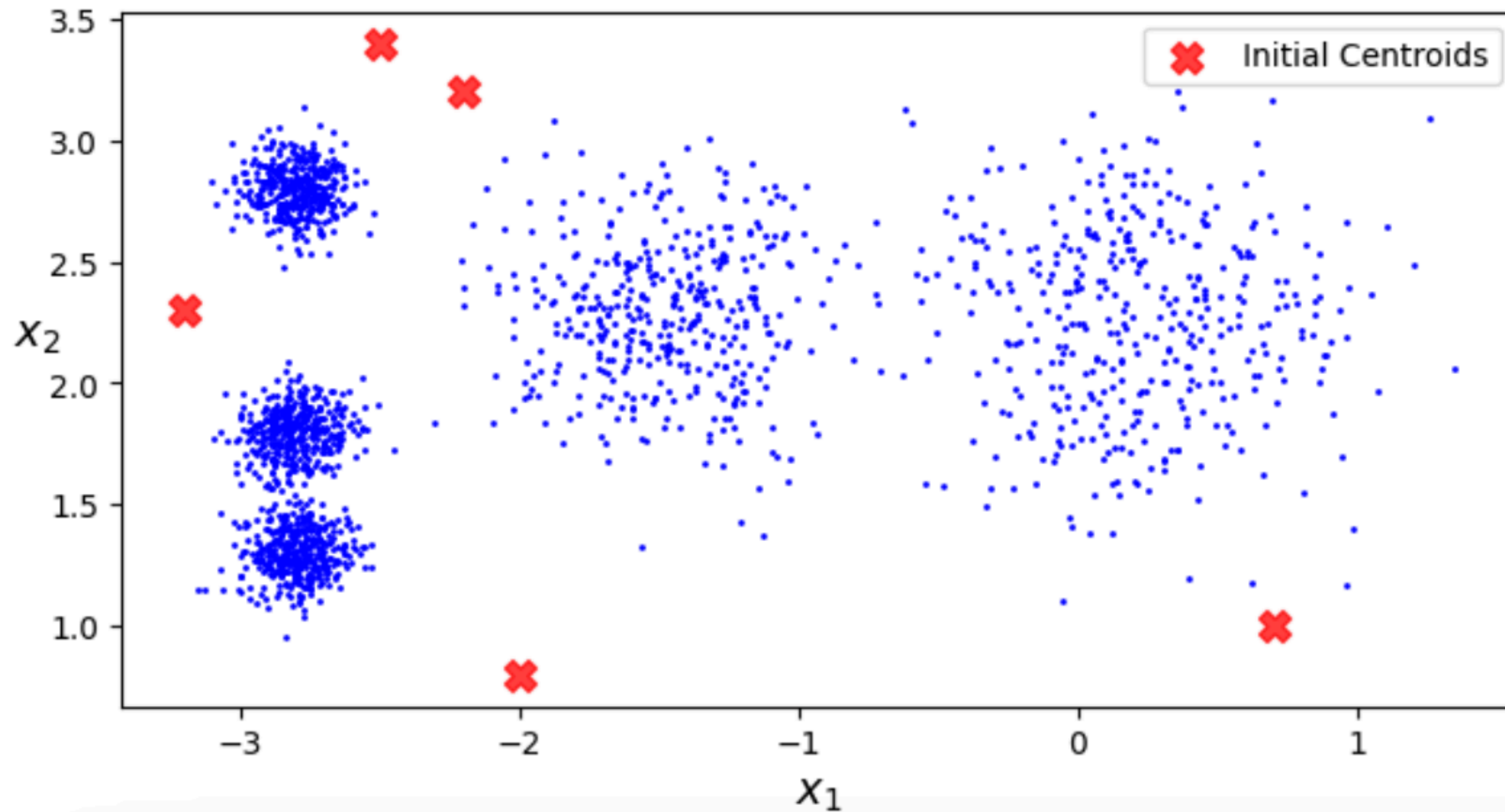
- Algoritmul împarte un set de date în grupuri (cluster), astfel încât punctele din același grup să fie cât mai asemănătoare între ele și cât mai diferite față de cele din alte grupuri
- Când folosim K-Means, trebuie să alegem câte grupuri dorim să formăm. Acest număr, „K”, este stabilit de noi (ex: $K=3$ înseamnă că vrem 3 cluster)

Clusterizarea K-Means

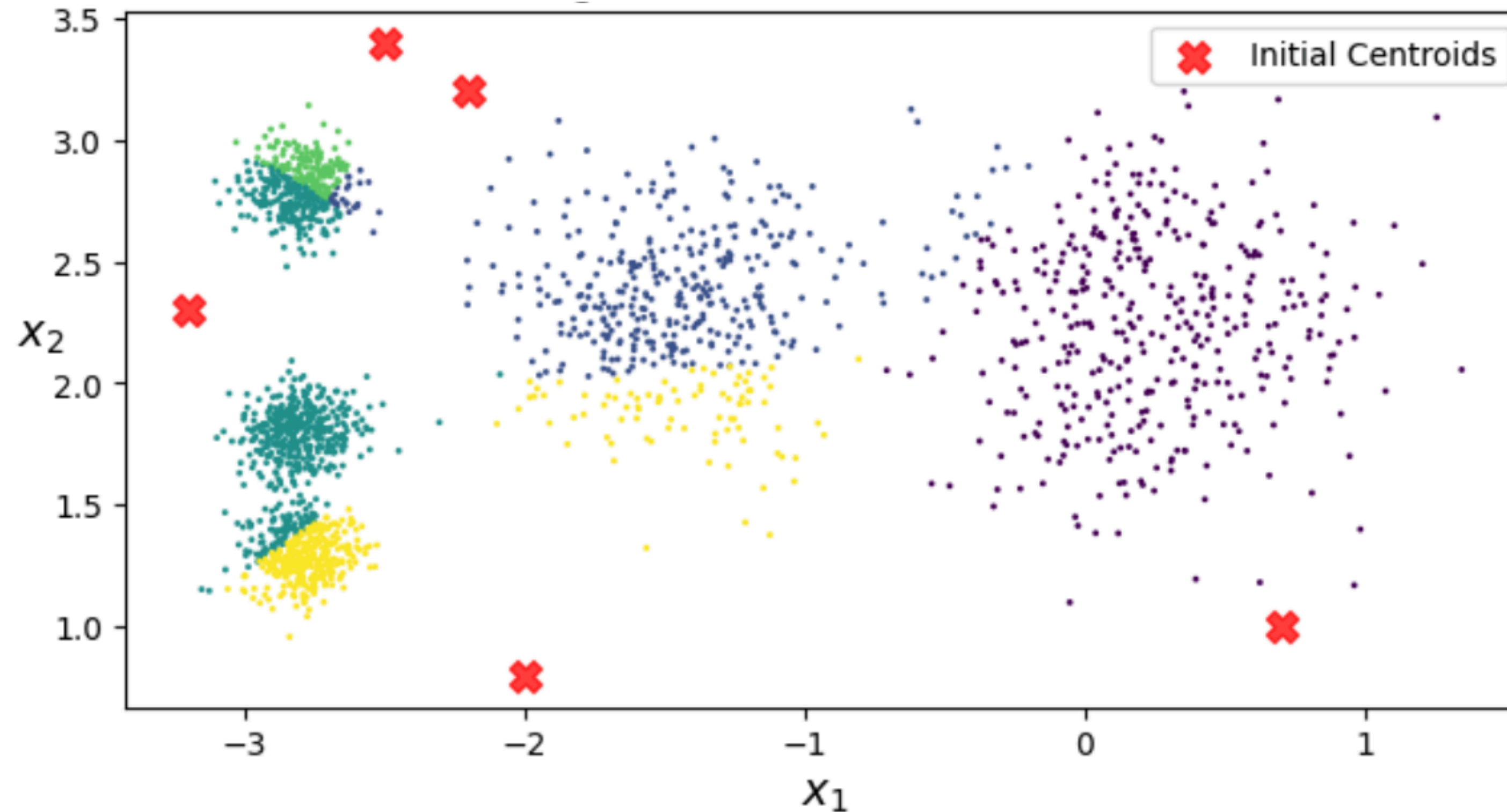


- Pasul 1: Algoritmul alege aleatoriu K puncte din date, care vor deveni „centroizii” inițiali (adică punctele centrale pentru fiecare cluster)

Clusterizarea K-Means

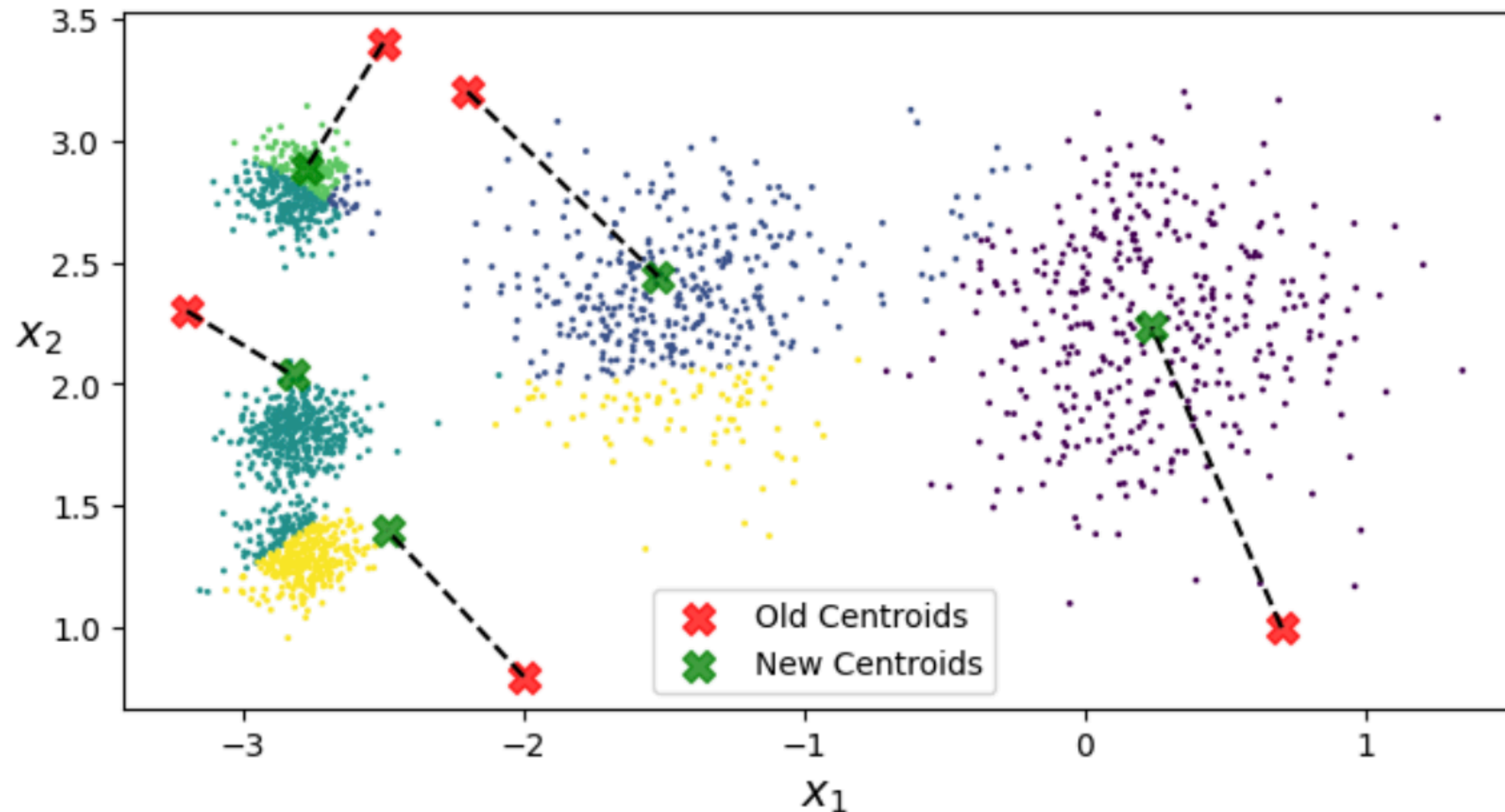


Clusterizarea K-Means



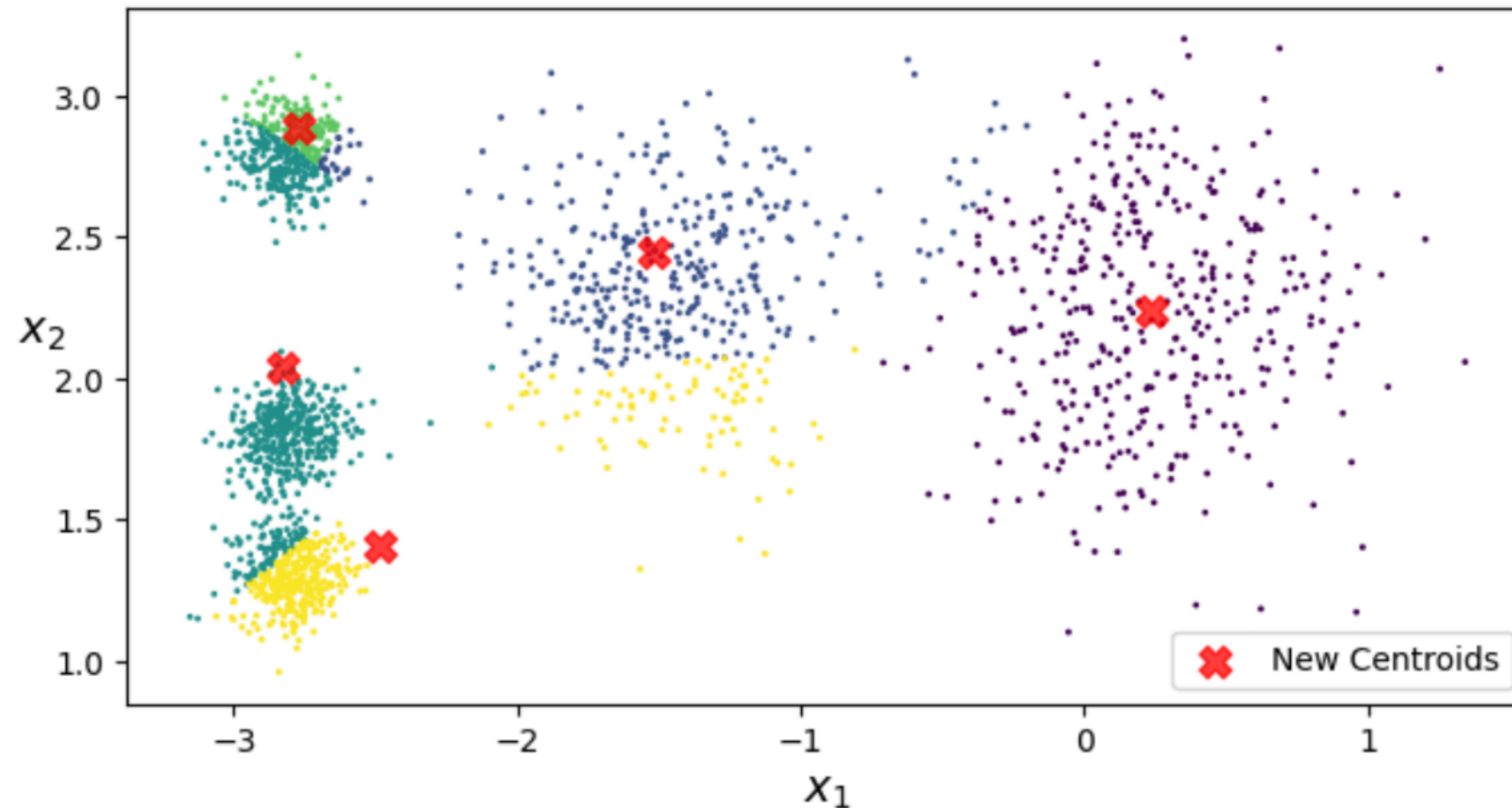
- Pasul 2: Fiecare punct din setul de date este alocat celui mai apropiat centroid (deci, grupurile încep să se formeze în jurul acestor centroizi).

Clusterizarea K-Means



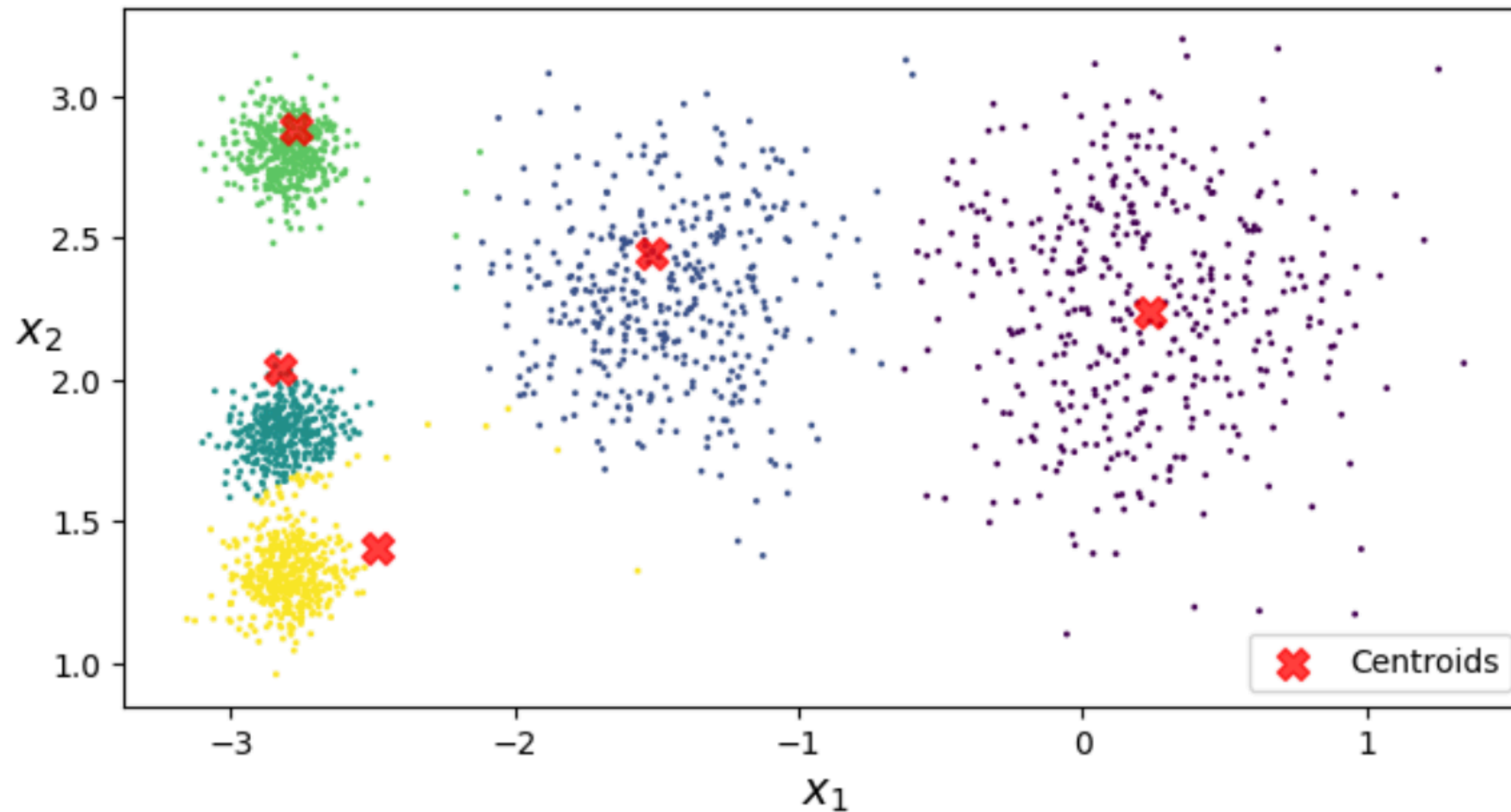
- Pasul 3: Odată ce fiecare punct a fost alocat unui cluster, se recalculază poziția fiecărui centroid (centrul fiecărui grup) ca fiind media punctelor grupului respectiv.

Clusterizarea K-Means



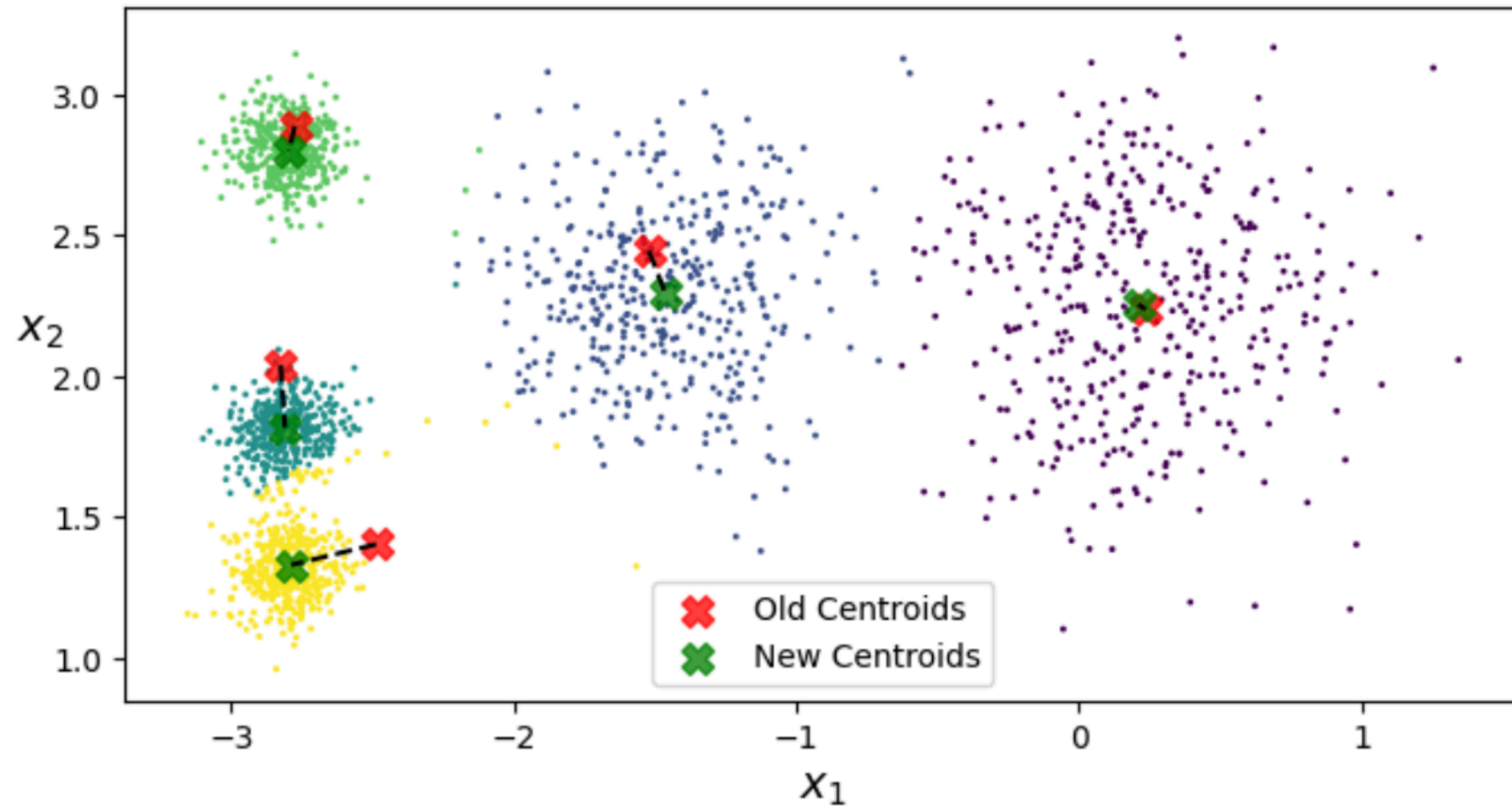
- Pasul 4: Repetăm pașii 2 și 3 până când centroizii nu se mai schimbă sau schimbările sunt minime. La final, fiecare punct va aparține unui cluster „stabil.”

Clusterizarea K-Means



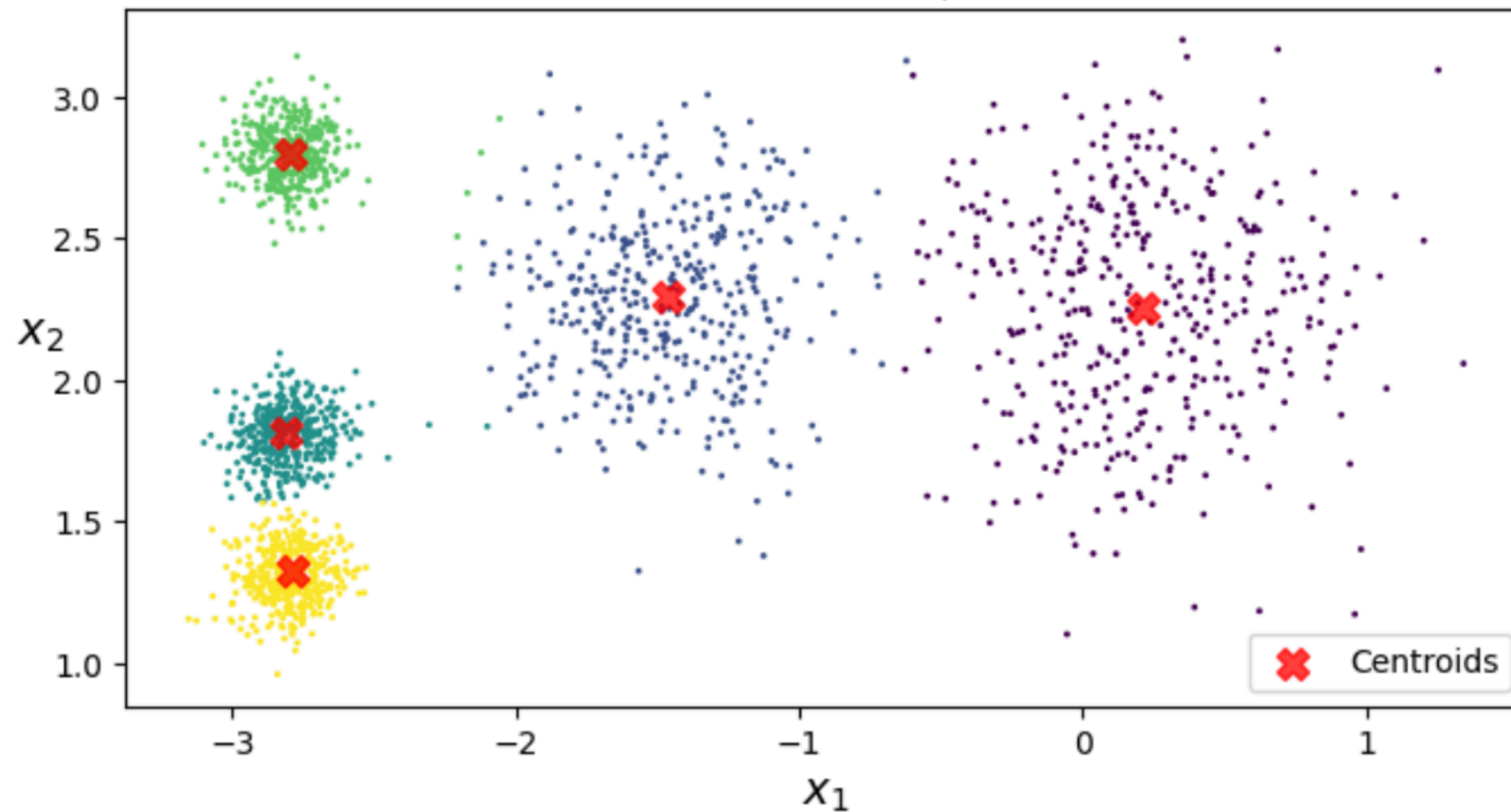
- Pasul 2: Fiecare punct din setul de date este alocat celui mai apropiat centroid (deci, grupurile încep să se formeze în jurul acestor centroizi).

Clusterizarea K-Means



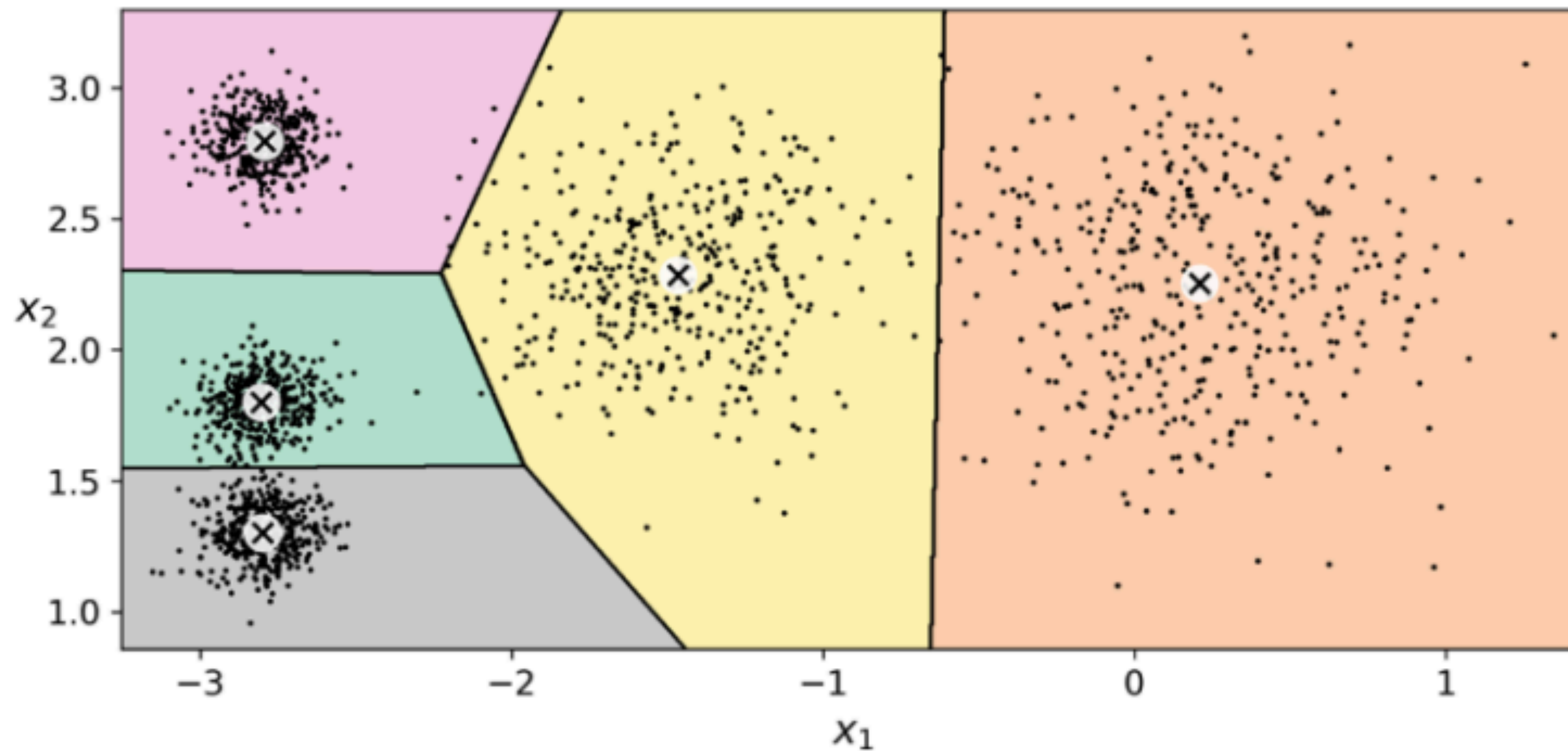
- Pasul 3: Odată ce fiecare punct a fost alocat unui cluster, se recalculază poziția fiecărui centroid (centrul fiecărui grup).

Clusterizarea K-Means

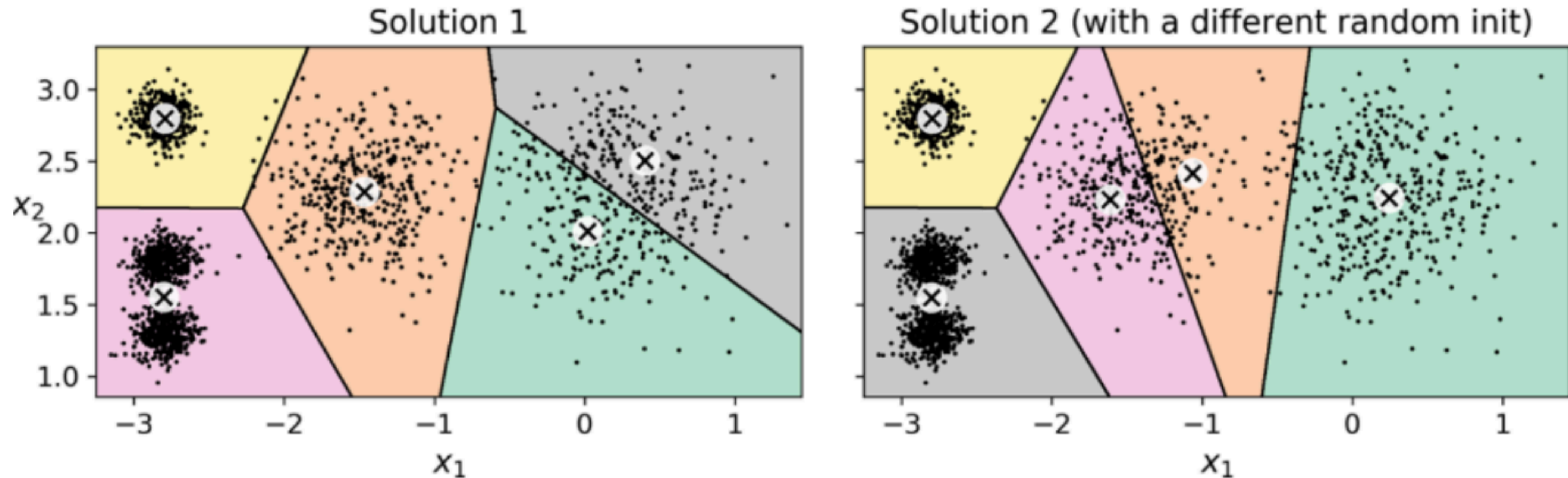


- În acest caz, două iterații au fost de ajuns pentru a obține niște centroizi optimi

Clusterizarea K-Means



Clusterizarea K-Means

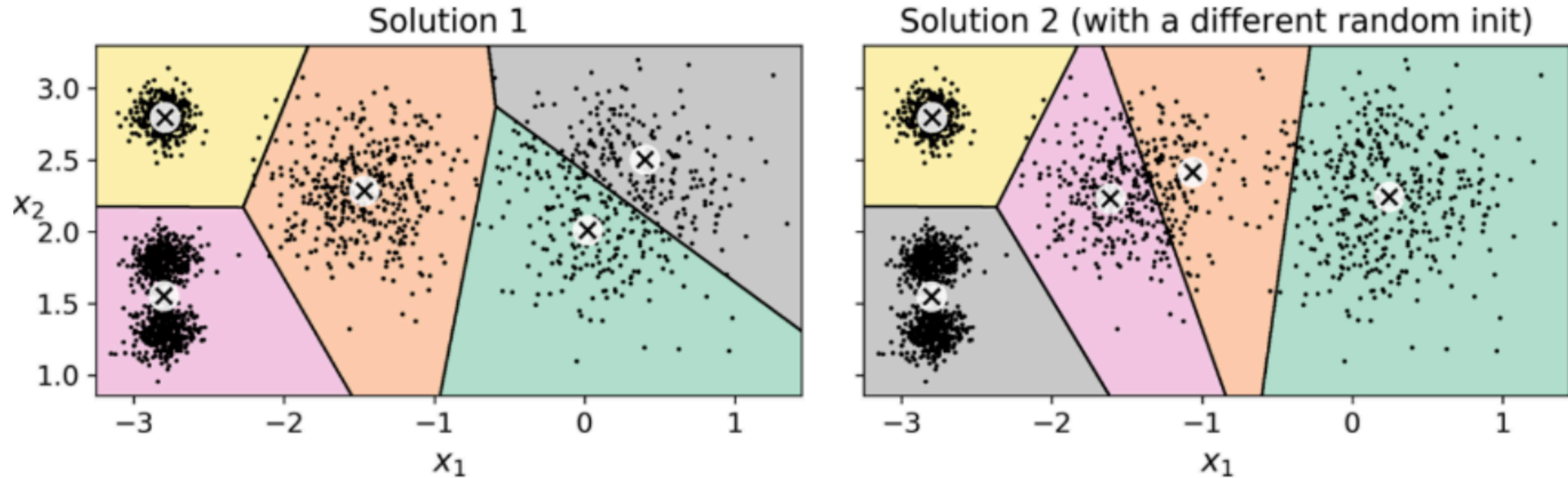


- **Problemă:** Convergența optimă a algoritmului depinde foarte mult de felul în care sunt aleși centrozii inițiali
- **Soluție:** Rularea algoritmului de mai multe ori cu diferiți centroizi și păstrarea rezultatului cel mai bun

Clusterizarea K-Means

- **Problemă:** Convergența optimă a algoritmului depinde foarte mult de felul în care sunt aleși centroizii inițiali
- **Soluție:** Rularea algoritmului de mai multe ori cu diferiți centroizi și păstrarea rezultatului cel mai bun
- Pentru aceasta, avem nevoie de o metodă prin care putem cuantifica cât de bun este un rezultat, adică cât de bine au fost identificate grupurile
- O astfel de metodă este calculul inerției pentru fiecare rezultat obținut
- Inerția reprezintă suma distanțelor fiecărui punct la centroidul asociat punctului respectiv

Clusterizarea K-Means



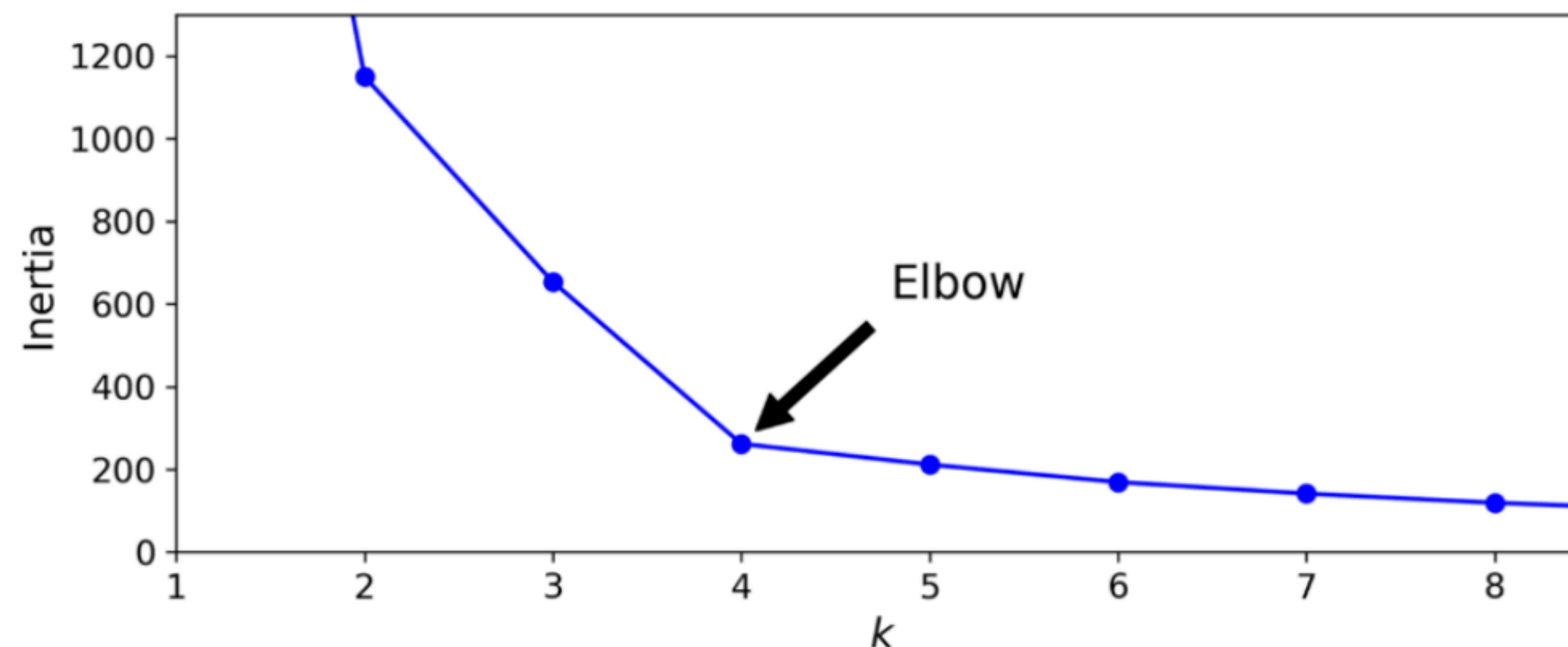
- Modelul din stânga prezintă o inerție de 223.3 pe când cel din dreapta are inerția 237.5
- Astfel, va fi ales modelul din stânga

Clusterizarea K-Means

- Totuși, cum știm să alegem K?
- Pentru datele trecute care au fost în 2D, totul a fost ușor pentru ca ochiometric le-am putut vizualiza și să identificăm numărul optim de clustere
- Problema apare atunci când avem mai multe dimensiuni, sau când clusterelor nu sunt evidente nici măcar ochiometric
- O metodă de alegere a lui K e să antrenăm mai multe modele cu $K = 1, 2, 3, \dots$, și să facem un grafic care reprezintă inerția fiecărui model în funcție de K

Clusterizarea K-Means

- O metodă de alegere a lui K e să antrenăm mai multe modele cu $K = 1, 2, 3, \text{ș.a.}$, și să facem un grafic care reprezintă inertăia fiecărui model în funcție de K



- În general, pe măsură ce crește K , inertia scade, dar după un anumit punct, reducerea devine tot mai mică. Forma graficului va avea o curbă, ca un cot
- Alegem valoarea lui K din „cotul” graficului, unde scăderea în valoarea erorii începe să se stabilizeze. Aceasta este metoda „elbow”. În acest caz, alegem $K = 4$

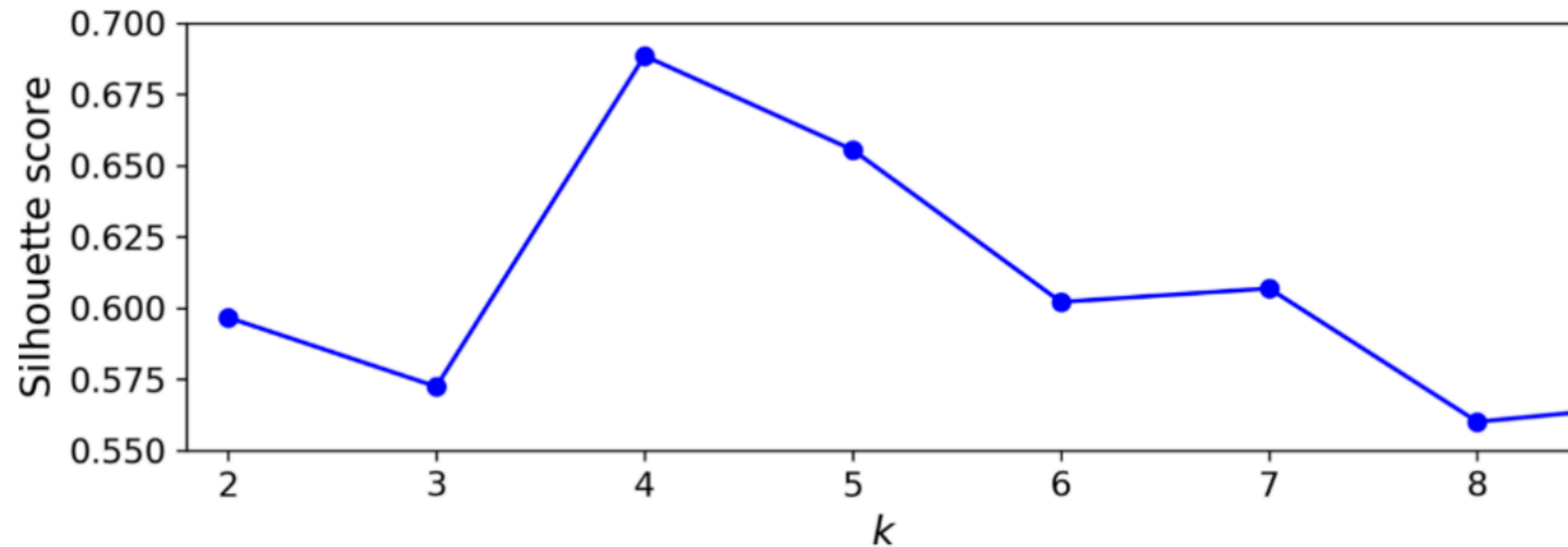
Clusterizarea K-Means

- De multe ori, „cotul” graficului Elbow poate fi neclar, iar interpretarea lui devine subiectivă
- Metoda Elbow doar măsoară reducerea erorii totale, dar nu indică direct calitatea separării fiecărui punct
- Metoda Silhouette ia în considerare cât de bine este plasat fiecare punct în interiorul clusterului său, comparând distanța medie față de punctele din clusterul cel mai apropiat
 - Un scor apropiat de 1 indică faptul că punctul este bine alocat clusterului său și este bine separat de alte cluster.
 - Un scor de aproximativ 0 indică faptul că punctul este la granița dintre două cluster
 - Un scor negativ (aproape de -1) arată că punctul este probabil alocat unui cluster greșit, fiind mai aproape de alt cluster

Clusterizarea K-Means

- Metoda Silhouette ia în considerare cât de bine este plasat fiecare punct în interiorul clusterului său, comparând distanța medie față de punctele din clusterul cel mai apropiat
 - Un scor apropiat de 1 indică faptul că punctul este bine alocat clusterului său și este bine separat de alte cluster.
 - Un scor de aproximativ 0 indică faptul că punctul este la granița dintre două cluster
 - Un scor negativ (aproape de -1) arată că punctul este probabil alocat unui cluster greșit, fiind mai aproape de alt cluster
- Calculăm media scorurilor Silhouette pentru toate punctele. Acest scor mediu reprezintă scorul Silhouette general și indică calitatea globală a modelului
- Cu cât scorul mediu este mai mare, cu atât modelul este mai bun

Clusterizarea K-Means



- Metoda Silhouette ne oferă o confirmare că $K = 4$ e o alegere bună
- De asemenea, prezintă niște rezultate calitative foarte bune: acum ne dăm seama că ce e după $K = 4$ s-ar putea să nu mai fie așa de bun
- Lucrul acesta nu era evident la metoda elbow

Clusterizarea K-Means

- Aplicarea K-Means original pe un set de date

```
0 sec.
from sklearn.datasets import make_moons
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split

X, y = make_moons(
    n_samples=1000,
    noise=0.1,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
clf = KMeans(n_clusters=2, n_init=1, algorithm='lloyd', init='random', random_state=42)
clf.fit(X_train)

clusters = clf.predict(X_test)

clusters[10], y_test[10], clf.labels_[16], clf.cluster_centers_, -clf.score(X_test), clf.score(X_train), clf.inertia_

(1,
0,
1,
array([[ -0.23292894,  0.58516198],
       [ 1.1880721 , -0.07686595]]),
86.63645014100341,
-329.87569181670045,
329.87569181670045)
```


Clusterizarea K-Means

```
clf = KMeans(n_clusters=2, n_init=1, algorithm='lloyd', init='random', random_state=42)
```

Parametri:

- **n_clusters**: câte grupuri să găsească modelul
- **n_init**: se vor antrena **n_init** modele și se va păstra modelul cel mai bun
- **algorithm**: dacă se folosește varianta originală (lloyd) sau, dacă e necesar, se folosește o valoare optimizată (care e și default)
- **init**: cum sunt inițializați centroizii pentru cele **n_init** modele
 - „random” le inițializează random
 - „k-means++” optimizează alegerea centroizilor la început (e default)

În loc să aleagă centrozii random, K-Means++ alege centrozii inițiali astfel încât aceștia să fie depărtați unul de altul, iar din experimente duce la rezultate mai bune

Clusterizarea K-Means

- Aplicarea K-Means original pe un set de date

```
0 sec.  from sklearn.datasets import make_moons
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split

X, y = make_moons(
    n_samples=1000,
    noise=0.1,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
clf = KMeans(n_clusters=2, n_init=1, algorithm='lloyd', init='random', random_state=42)
clf.fit(X_train)

clusters = clf.predict(X_test)

clusters[10], y_test[10], clf.labels_[16], clf.cluster_centers_, -clf.score(X_test), clf.score(X_train), clf.inertia_

 (1,
0,
1,
array([[ -0.23292894,  0.58516198],
       [ 1.1880721 , -0.07686595]]),
86.63645014100341,
-329.87569181670045,
329.87569181670045)
```

Clusterizarea K-Means

Afișez:

- Prezicerea celui de al 11-lea exemplar din setul de test
- Eticheta adevărată (ground truth) a celui de al 11-lea exemplar din setul de test
- În ce cluster se situează al 17-lea exemplar din setul de train
- Coordonatele asociate centroizilor
- Inerția pe setul de test
- Inerția negativă pe setul de train
- Inerția pe setul de train

```
0 sec.
from sklearn.datasets import make_moons
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split

X, y = make_moons(
    n_samples=1000,
    noise=0.1,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
clf = KMeans(n_clusters=2, n_init=1, algorithm='lloyd', init='random', random_state=42)
clf.fit(X_train)

clusters = clf.predict(X_test)

clusters[10], y_test[10], clf.labels_[16], clf.cluster_centers_, -clf.score(X_test), clf.score(X_train), clf.inertia_

(1,
0,
1,
array([[ -0.23292894,  0.58516198],
       [ 1.1880721 , -0.07686595]]),
86.63645014100341,
-329.87569181670045,
329.87569181670045)
```


Clusterizarea K-Means

clf = KMeans(n_clusters=2, n_init=1, algorithm='lloyd', init='random', random_state=42)

- KMeans folosește tot setul de date dat la metoda fit() ca să actualizeze centroizii într-un pas de iterare
- S-a creat MiniBatchKMeans: acesta folosește doar un subset din date pentru a actualiza centroizii

clf = MiniBatchKMeans(n_clusters=2, batch_size = 50, n_init=1, init='random', random_state=42)

- batch_size: câte date să folosească la actualizare
- lipsește parametrul algorithm: va folosi el o varianta optimă
- MiniBatchKMeans e util la seturi de date foarte mari

Clusterizarea K-Means

```
✓ 0 sec. ▶ from sklearn.datasets import make_moons
from sklearn.cluster import MiniBatchKMeans
from sklearn.model_selection import train_test_split

X, y = make_moons(n_samples=1000, noise=0.1, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = MiniBatchKMeans(n_clusters=2, n_init=10, batch_size=50, init='k-means++', random_state=42)
clf.fit(X_train)

clusters = clf.predict(X_test)

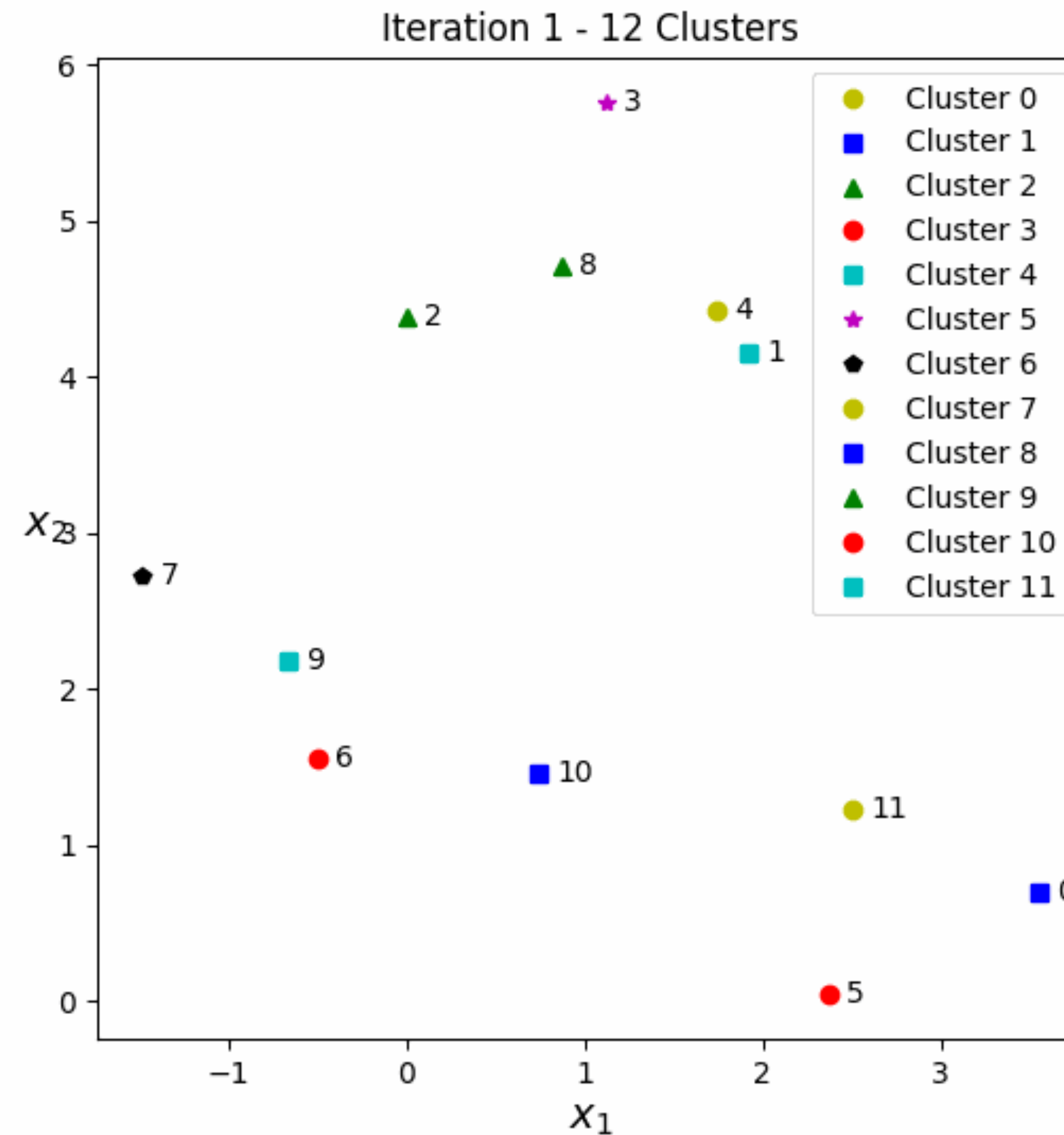
clusters[10], y_test[10], clf.labels_[16], clf.cluster_centers_, -clf.score(X_test), clf.score(X_train), clf.inertia_

(0,
0,
0,
array([[ 1.14988231, -0.07348405],
        [-0.28341901,  0.63970077]]),
88.04307483029518,
-331.70820139536573,
331.70820139536573)
```

- Aici folosim MiniBatchKMeans, cu o actualizare a centroizilor bazându-se pe 50 de puncte de date și folosind optimizarea K-Means++. Se antrenează 10 modele și se alege cel mai bun dintre ele

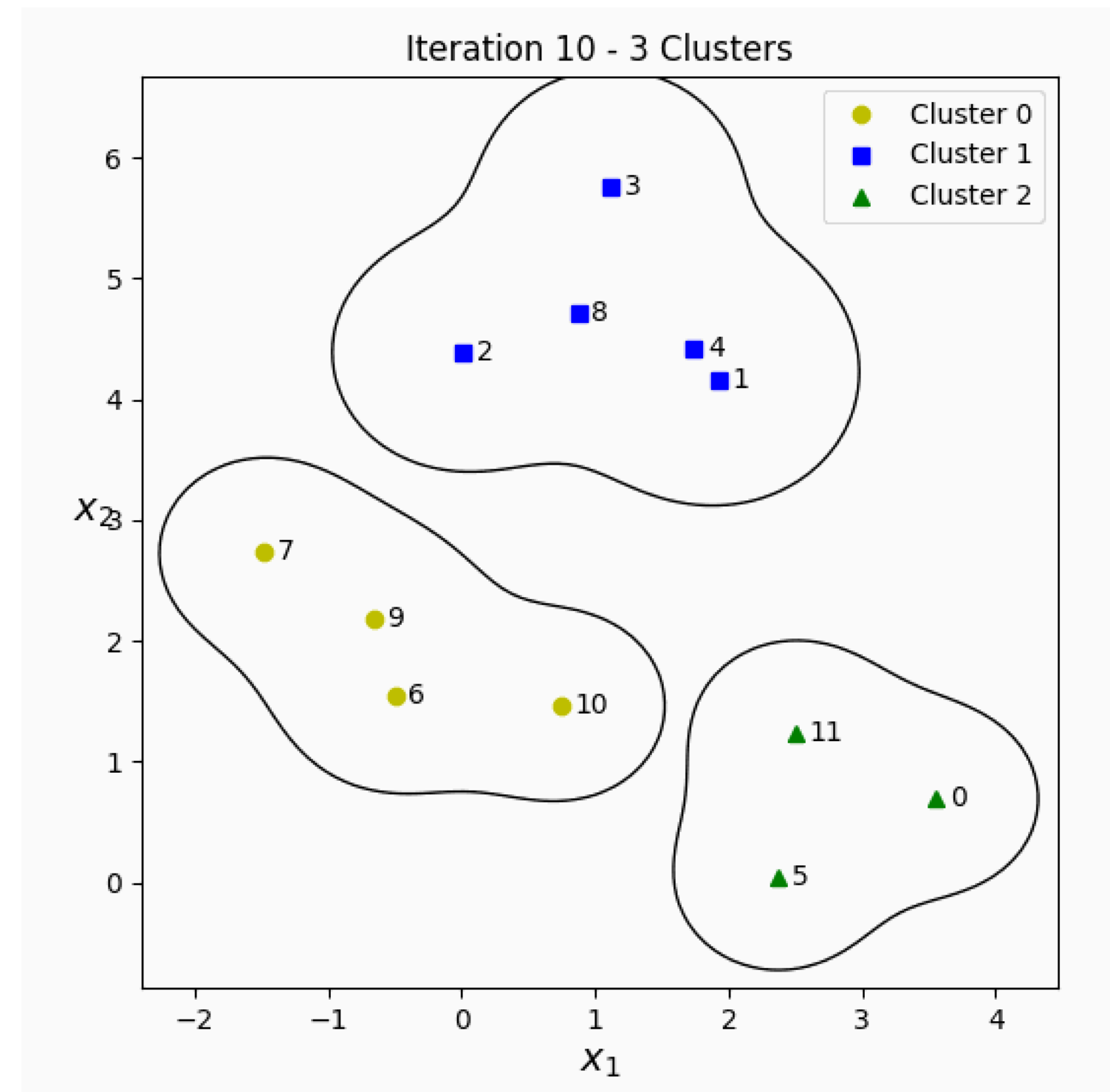
- Aglomerative Clustering prezintă o altă metodă de a găsi grupuri în date
- Metoda începe prin a asocia un cluster către fiecare punct, deci fiecare punct va reprezenta inițial un cluster
- Apoi, în fiecare iterație, va calcula distanța între fiecare cluster și va contopi clusterelor care au distanță minimă (le va face un singur cluster)
- Algoritmul continuă până când s-a ajuns la un singur cluster mare sau până s-a ajuns la o condiție de stop (de exemplu, numărul maxim de cluster)

Aglomerative Clustering



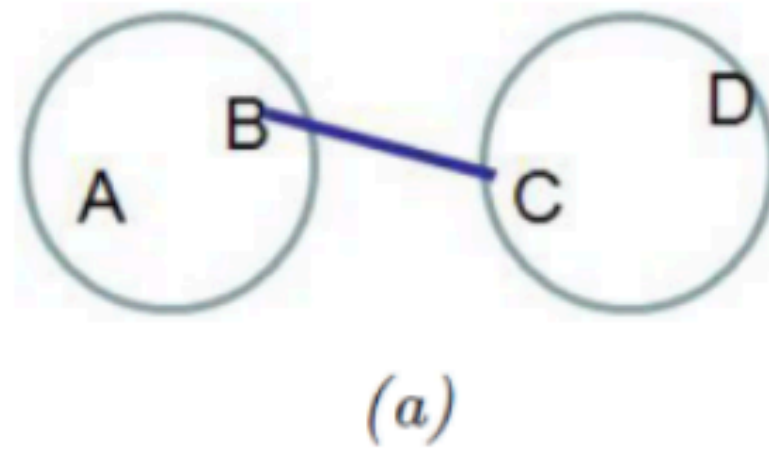
Aglomerative Clustering

- E ușor să calculăm distanța între două puncte, dar cum definim distanța dintre două cluster?

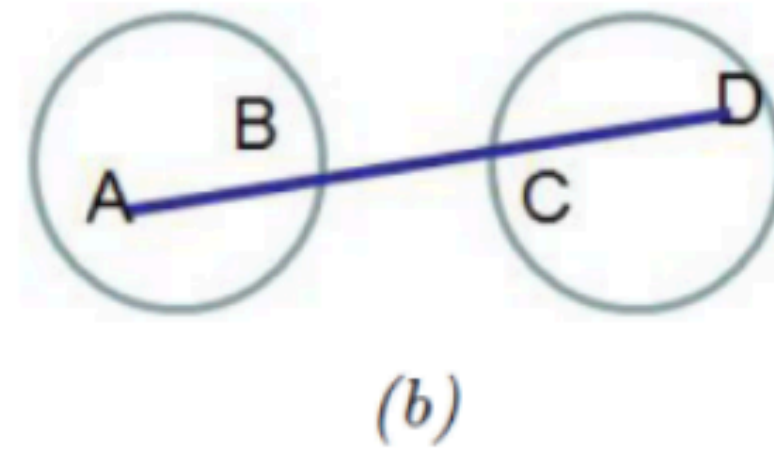


- De exemplu, în iterația 10 am ajuns la 3 cluster, cum știm care cluster e mai aproape de altul?

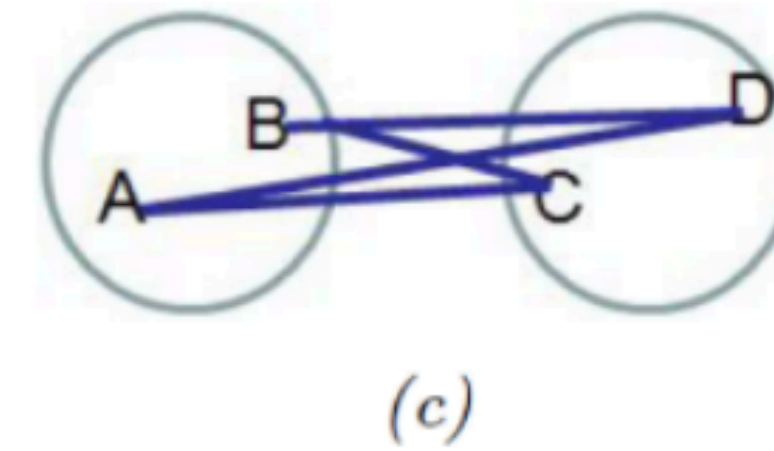
Aglomerative Clustering



Single Linkage



Complete Linkage



Average Linkage

- S-au definit 3 metode de calcul al distanțelor dintre cluster:

Single Linkage: distanța dintre două cluster este definită ca distanța dintre cele mai apropriate elemente dintre cele două cluster

Complete Linkage: distanța dintre două cluster este definită ca distanța dintre cele mai depărtate elemente dintre cele două cluster

Average Linkage: distanța dintre două cluster este definită ca media distanțelor dintre toate perechile de elemente din cele două cluster

Aglomerative Clustering

```
▶ from sklearn.cluster import AgglomerativeClustering
X, y = make_blobs(random_state=0, n_samples=12)

agg = AgglomerativeClustering(n_clusters=3, linkage="average")
clusters = agg.fit_predict(X)

clusters

array([1, 2, 2, 2, 2, 1, 0, 0, 2, 0, 0, 1])
```

- Afișăm în ce cluster a fost asociat fiecare element din setul de date

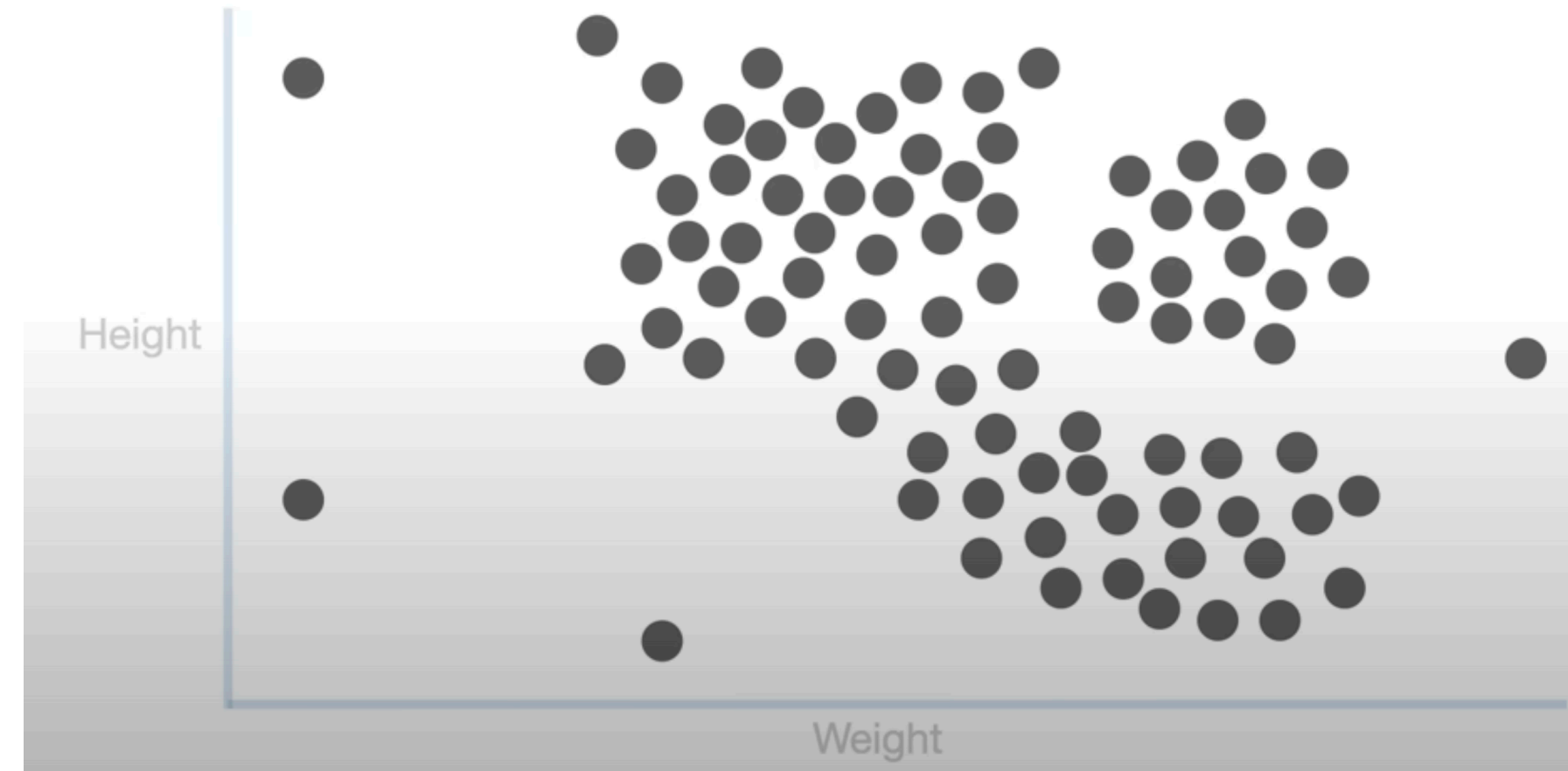
Aglomerative Clustering

```
agg = AgglomerativeClustering(n_clusters=3, linkage="average")
```

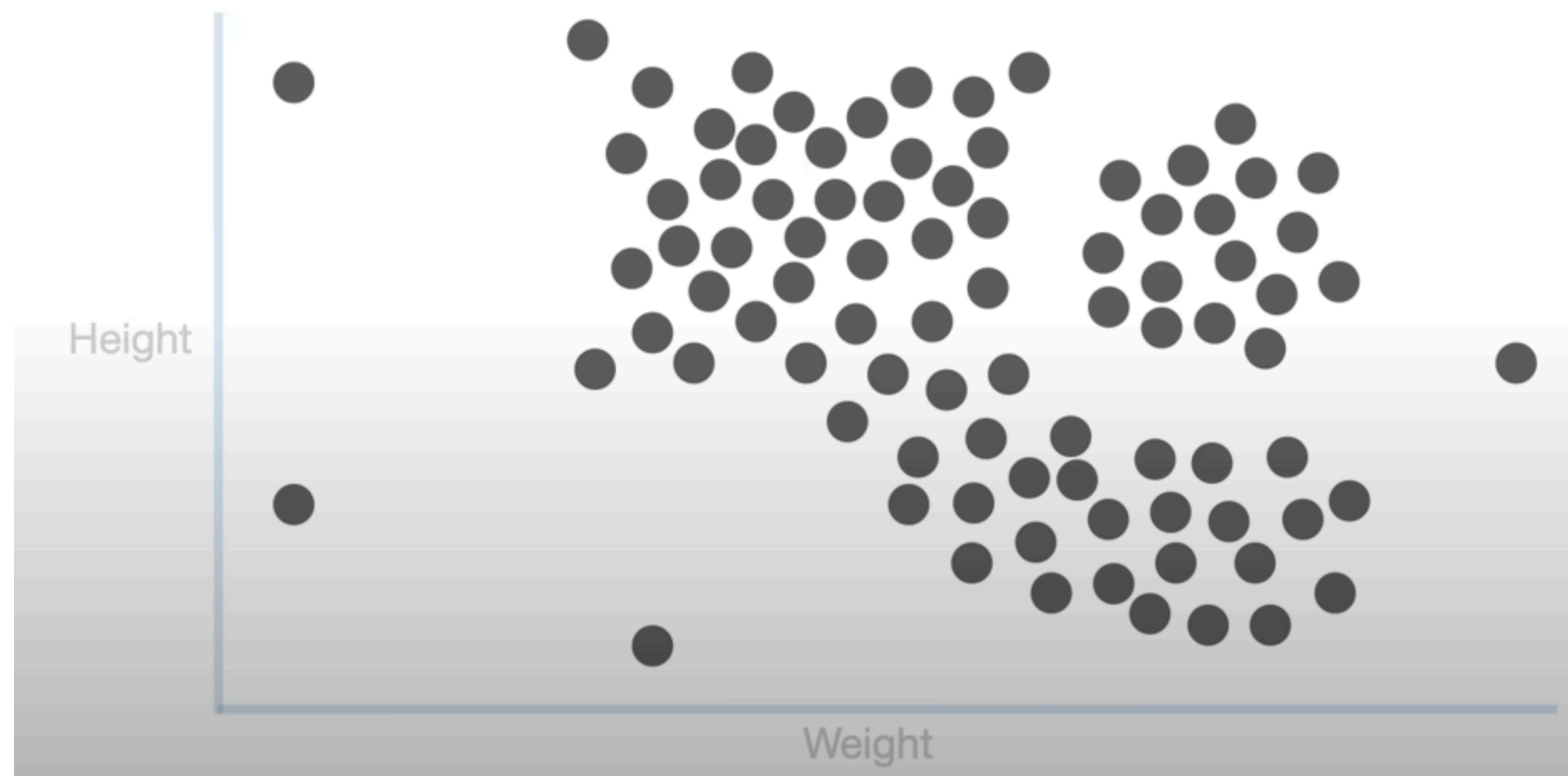
Parametri:

- `n_clusters`: numărul final de grupuri
- `linkage`: primește unul dintre valorile „single”, „complete” sau „average”

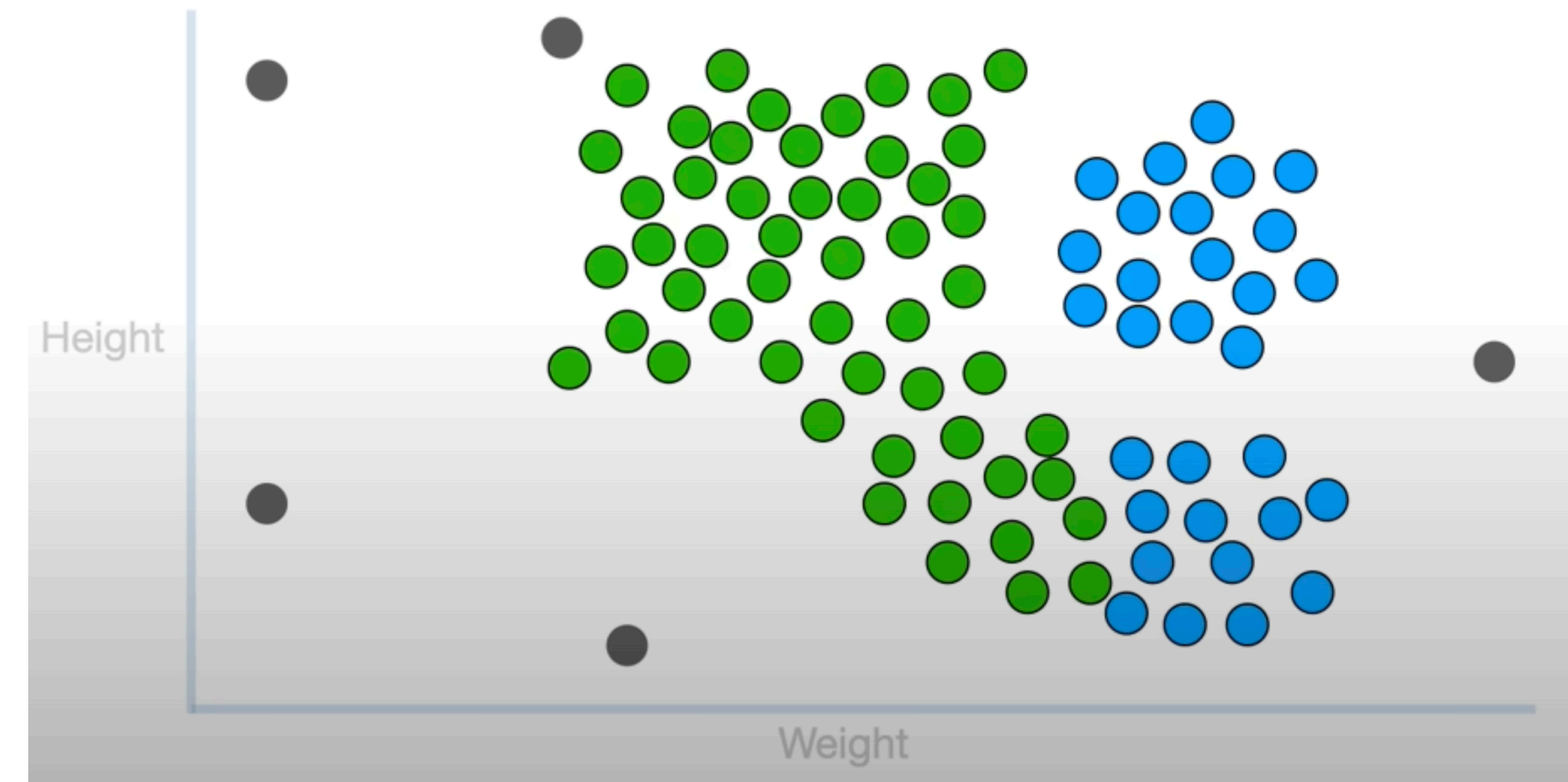
- Altă metodă de clusterizare este DBSCAN
- Oamenii își pot da seama de clustere pe baza a cât de dense sunt niște grupuri de puncte



- Aici, e ușor să ne dăm seama că exista 2 clustere
- Punctele izolate sunt în zone cu densitate mică s-ar putea să fie outliers, adică măsurători greșite

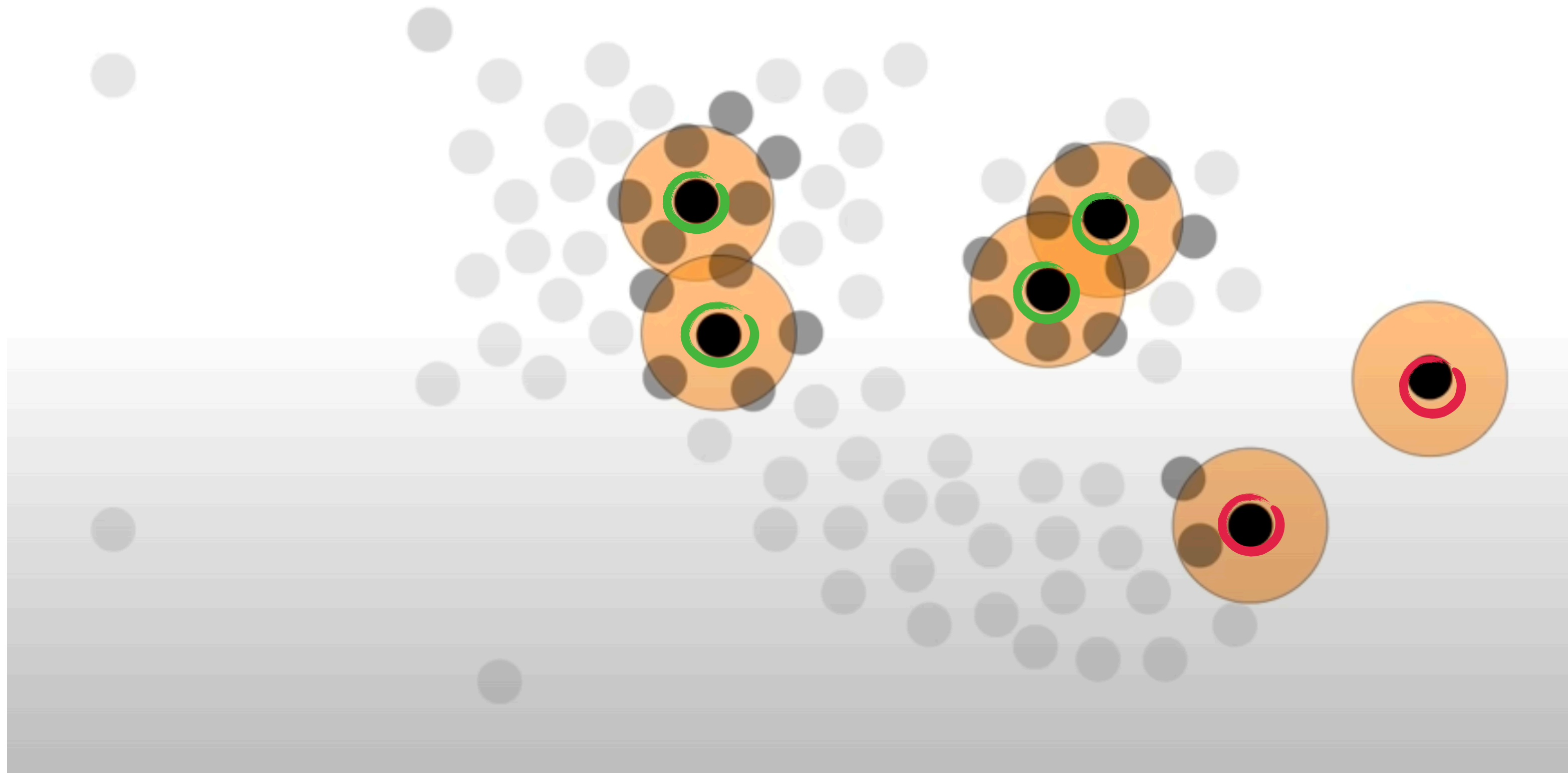


Setul de date

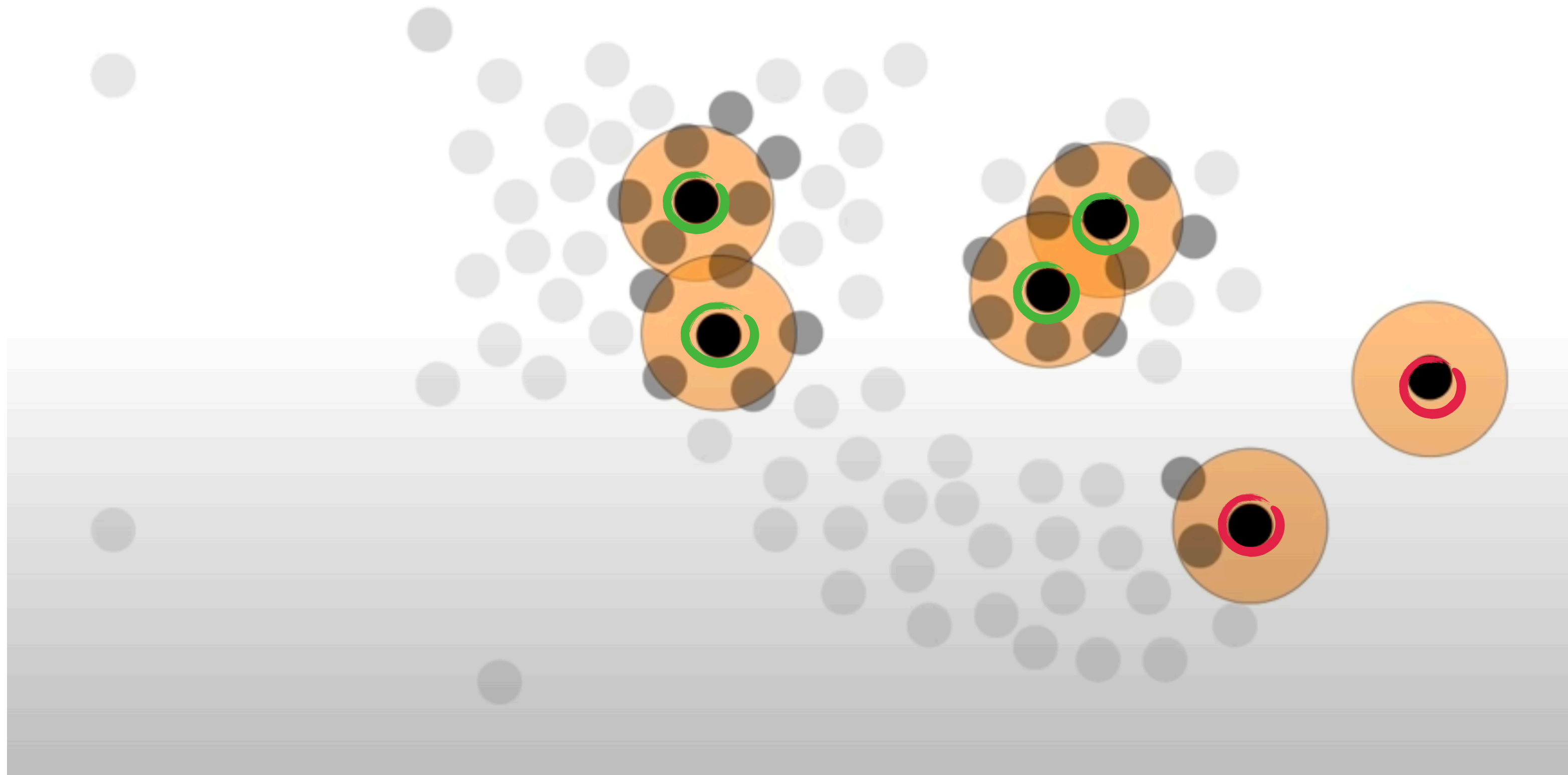


Aplicare K-Means

- K-Means și Aglomerative Clustering nu fac o treabă foarte bună atunci când clusterelor sunt destul de apropiate între ele

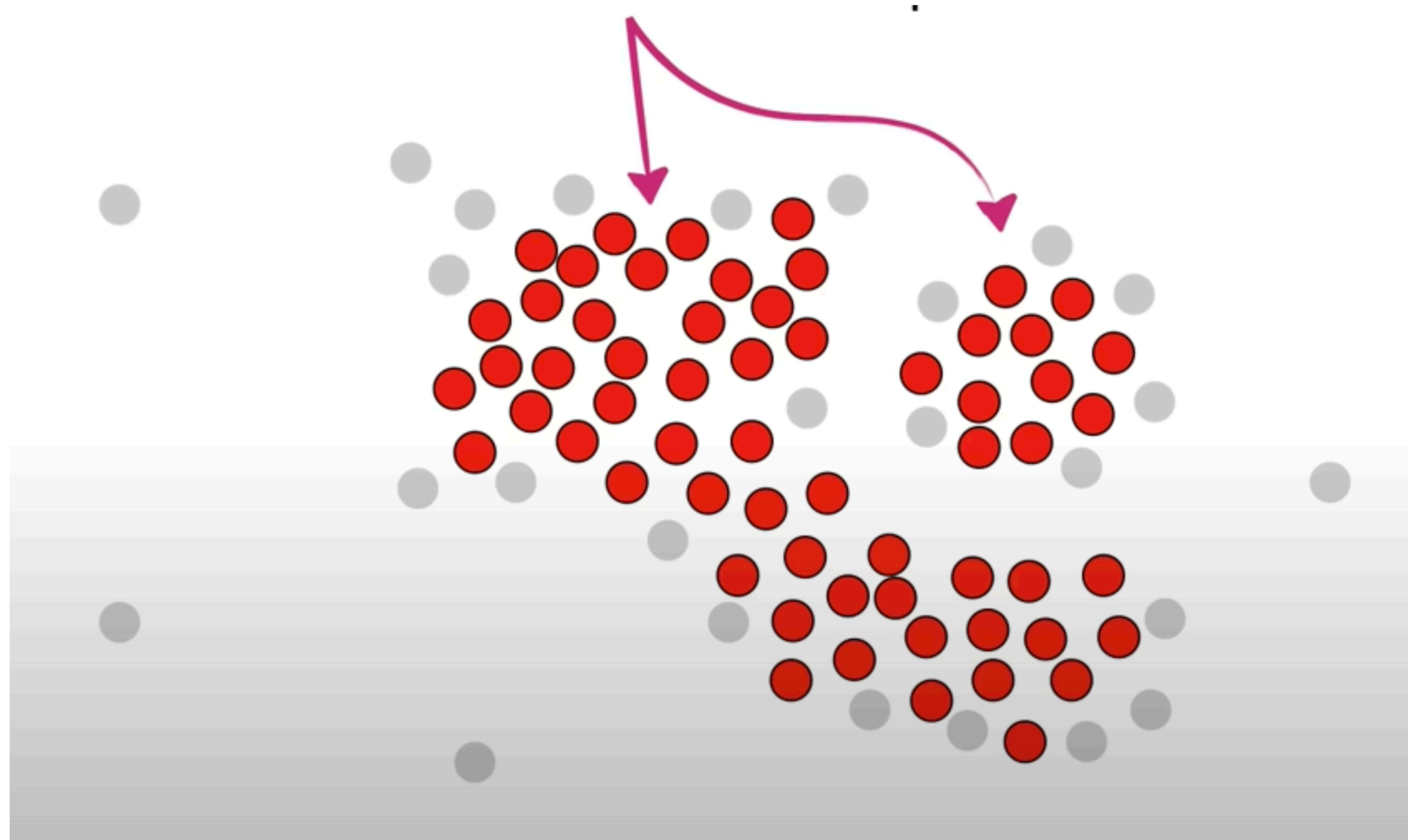


- DBSCAN se uită la o rază ϵ de fiecare punct dacă există cel puțin MinPts puncte apropiate de el în acea rază (în exemplu presupunem minPts=4)
- Dacă punctul are cel puțin MinPts vecini într-o rază ϵ , acel punct este **core point**
- Altfel, este un **punct non-core**

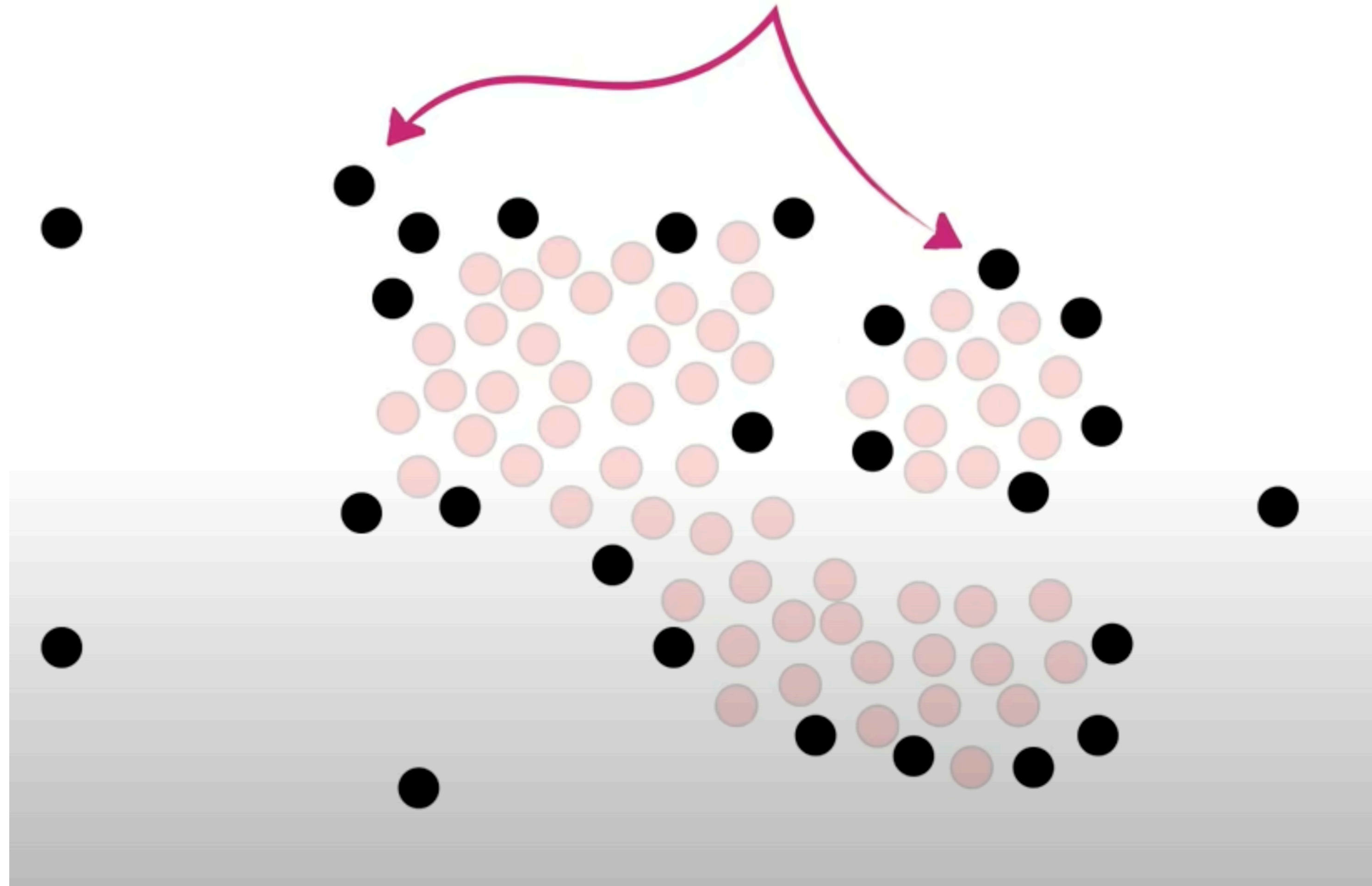


- Algoritmul face asta într-un mod iterativ: alege un punct random, găsește vecinii, dacă are cel puțin MinPts vecini, atunci vecinii lui vor fi folosiți pentru a fi verificate în continuare dacă sunt core points și tot așa

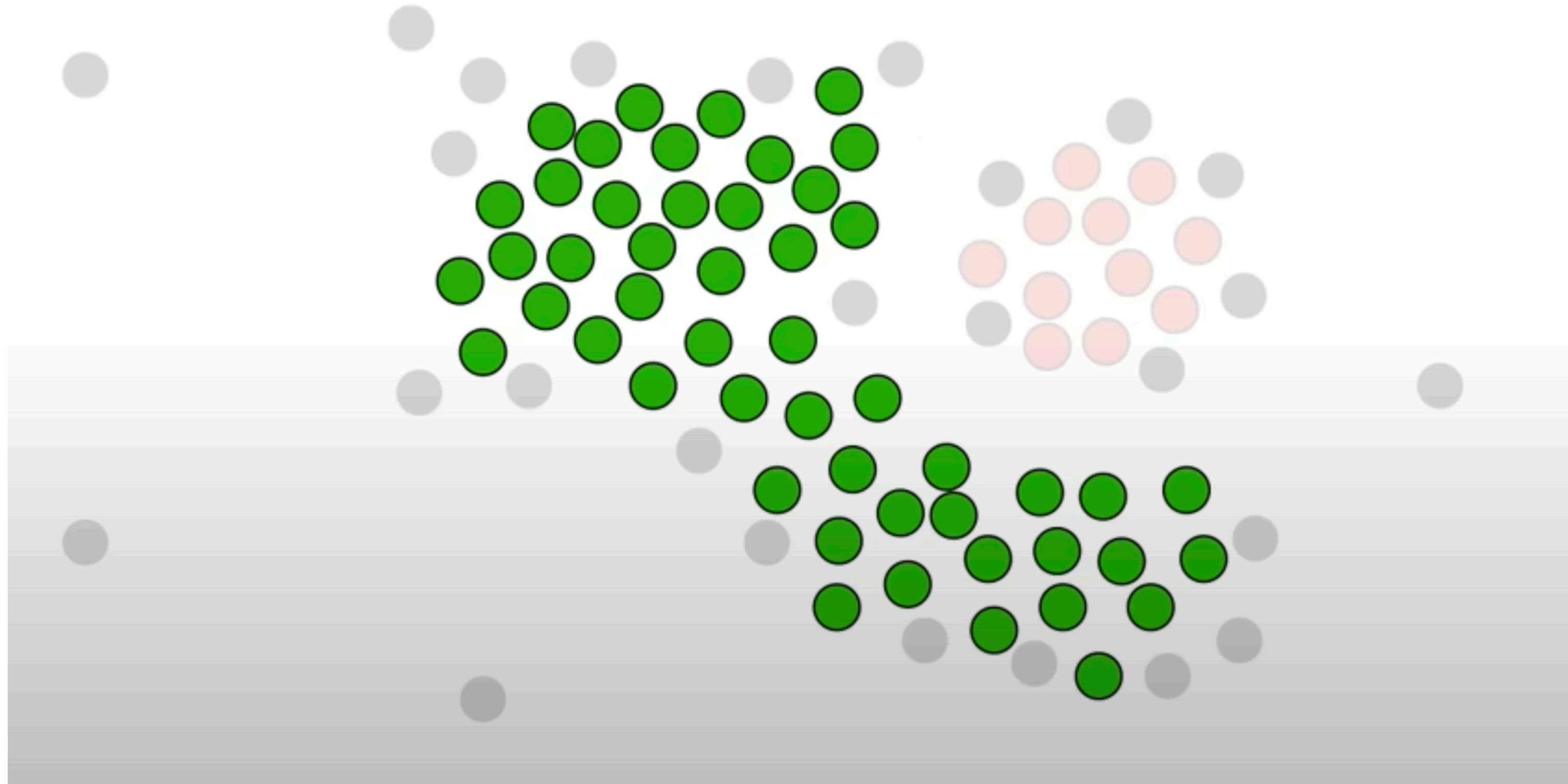
- Putem spune că toate acestea sunt core points, deoarece în raza lor există cel puțin MinPts puncte



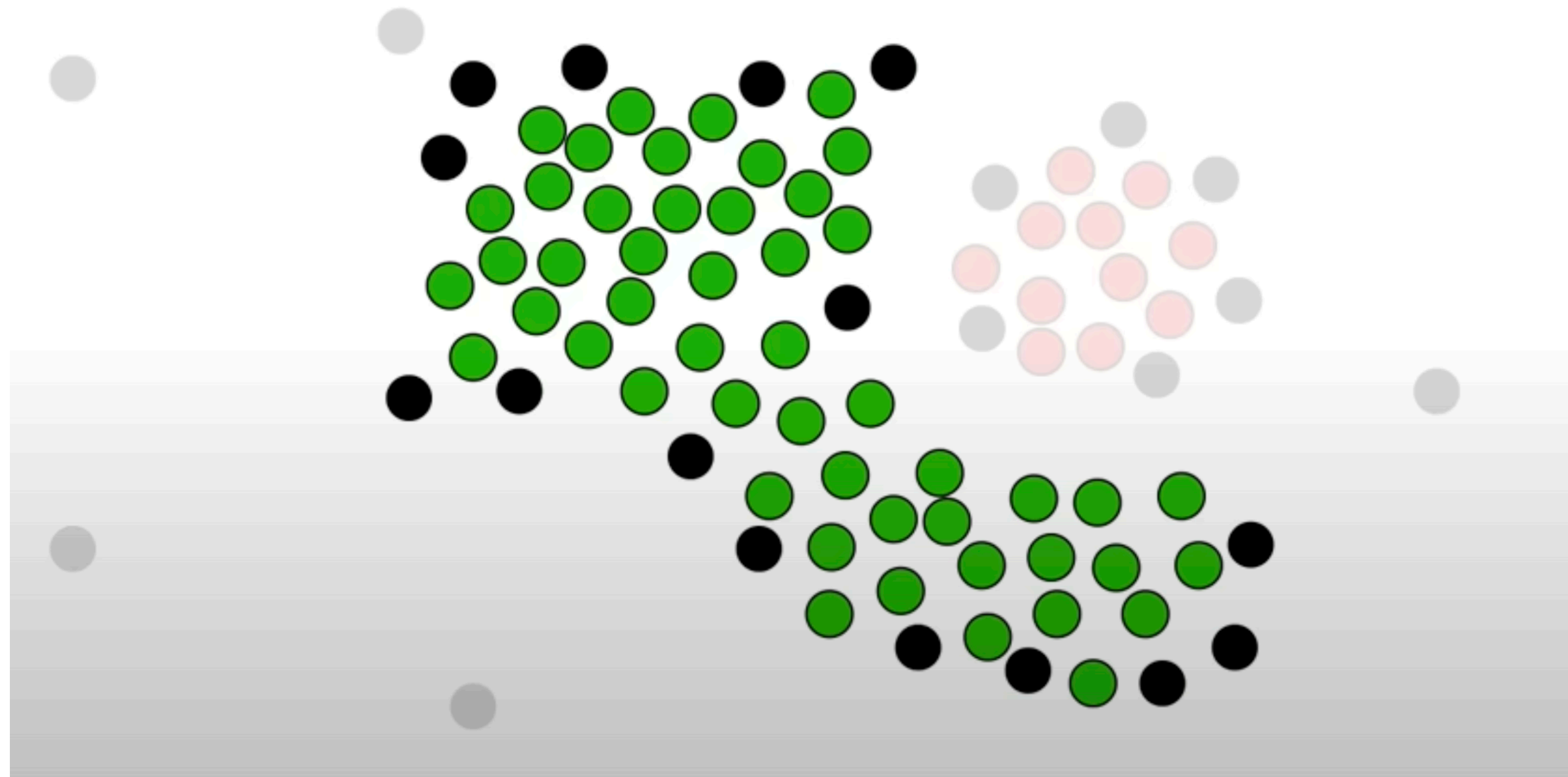
- Mai mult, aceste puncte sunt non-core, deoarece în raza lor nu se regăsesc cel puțin MinPts puncte



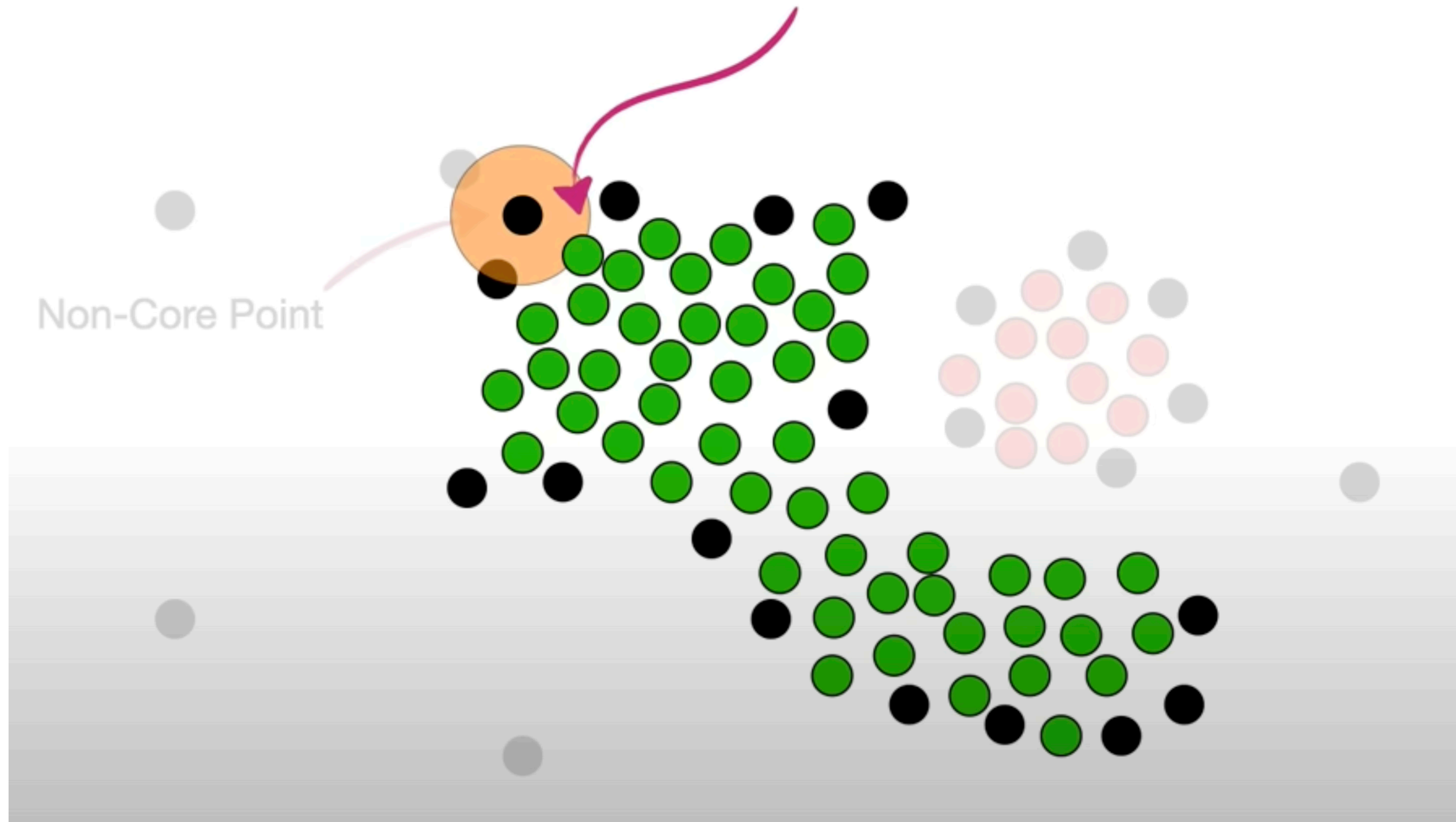
- Deci, toate aceste puncte vor forma un cluster, deoarece sunt core points



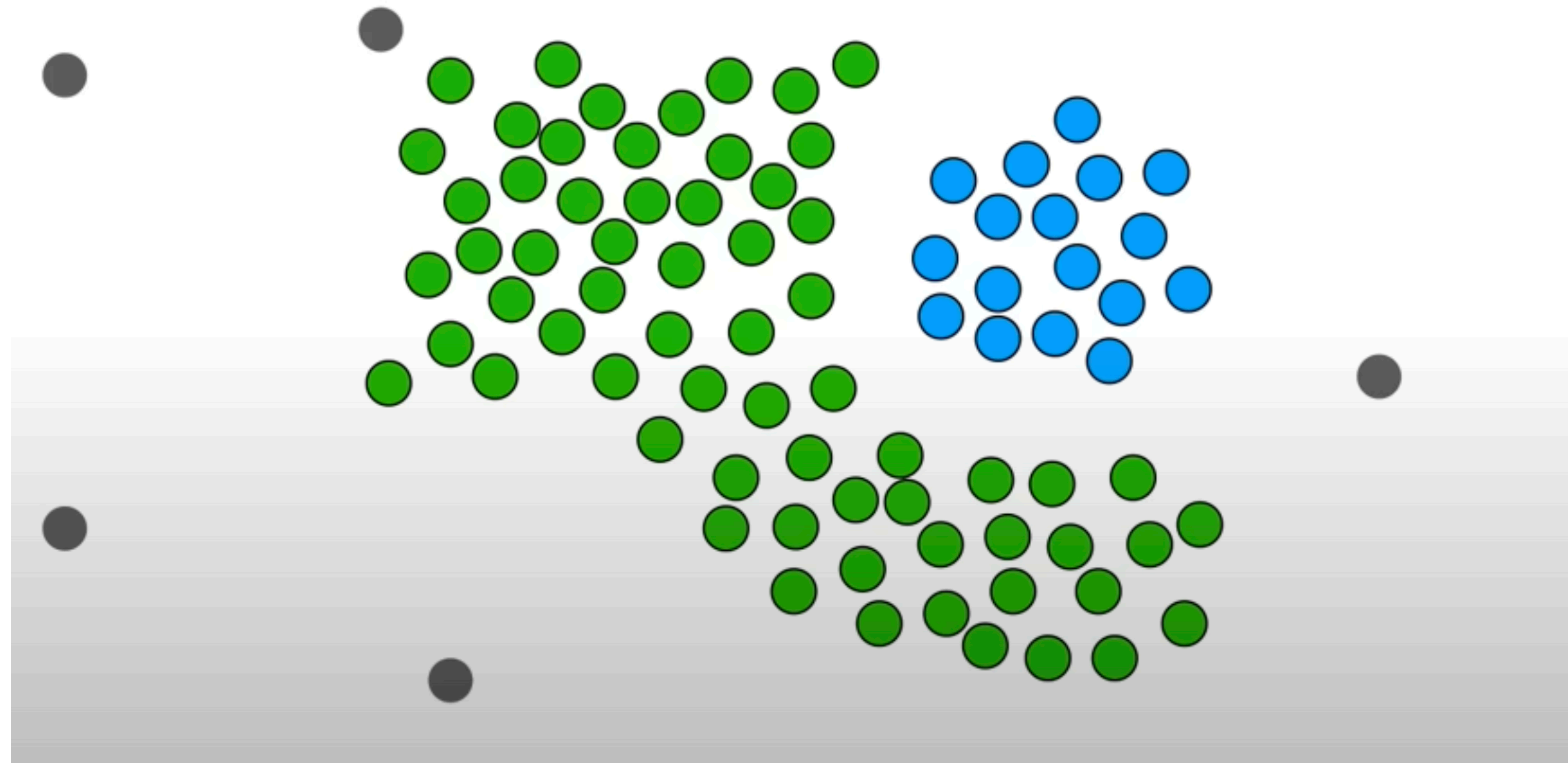
- Trebuie să facem ceva în legătură cu punctele la granițe
- Soluția este să adăugăm acele puncte non-core în clusterul 1 dacă ele sunt apropiate de cel puțin un core point din acel cluster



- De exemplu acest punct, care e non-core, este aproape de un core-point, așa că îl considerăm la clusterul 1
- La fel vom proceda cu toate celelalte puncte de la graniță



- După ce folosim DBSCAN, ajungem la ceva de genul
- Punctele care nu sunt în niciun cluster vor fi numite outliers / zgomote / erori de măsurare



- DBSCAN consideră că un cluster e o zonă densă de puncte. Pentru a decide ce înseamnă “dens”, folosește două concepte:
 - Epsilon (ϵ): raza în jurul unui punct în care se caută altele pentru a forma un cluster
 - MinPts: numărul minim de puncte necesar într-o rază ϵ pentru a considera că e un “cluster”
- Tipuri de puncte:
 - Core points: au cel puțin MinPts vecini într-o rază ϵ (sunt punctele de bază ale clusterului)
 - Border points: au mai puțini vecini decât MinPts, dar sunt în raza ϵ a unui core point
 - Noise points sau outliers: puncte care nu fac parte din niciun cluster, fiind prea departe de orice zonă densă.
- DBSCAN începe cu un punct oarecare și vede dacă e un core point (are destui vecini în raza ϵ). Dacă da, se extinde în cluster adăugând vecinii direcți și verificând dacă aceștia sunt la rândul lor core points, și tot așa

✓
0
sec.

```
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN

X, y = make_moons(n_samples=1000, noise=0.05, random_state=42)

dbscan = DBSCAN(eps=0.05, min_samples=5)
dbscan.fit(X)

dbscan.labels_[:10], len(dbscan.core_sample_indices_), dbscan.core_sample_indices_[:10], dbscan.components_
```

```
(array([ 0,  2, -1, -1,  1,  0,  0,  0,  2,  5]),
808,
array([ 0,  4,  5,  6,  7,  8, 10, 11, 12, 13]),
array([[ -0.02137124,  0.40618608],
       [-0.84192557,  0.53058695],
       [ 0.58930337, -0.32137599],
       ...,
       [ 1.66258462, -0.3079193 ],
       [-0.94355873,  0.3278936 ],
       [ 0.79419406,  0.60777171]]))
```

Parametri:

- eps: raza după care se caută vecini
- min_samples: câte puncte trebuie să fie la o rază de eps de un punct ca să fie considerat core sau nu

```
0 sec.
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN

X, y = make_moons(n_samples=1000, noise=0.05, random_state=42)

dbscan = DBSCAN(eps=0.05, min_samples=5)
dbscan.fit(X)

dbscan.labels_[:10], len(dbscan.core_sample_indices_), dbscan.core_sample_indices_[:10], dbscan.components_

(array([ 0,  2, -1, -1,  1,  0,  0,  0,  2,  5]), 808,
 array([ 0,  4,  5,  6,  7,  8, 10, 11, 12, 13]),
 array([[ -0.02137124,  0.40618608],
        [ -0.84192557,  0.53058695],
        [  0.58930337, -0.32137599],
        ...,
        [  1.66258462, -0.3079193 ],
        [ -0.94355873,  0.3278936 ],
        [  0.79419406,  0.60777171]]))
```

Afișez:

- Primele 10 etichete ale clusterelor atribuite fiecărui punct din setul de date (-1 e outlier)
- Numărul total de core points identificate de DBSCAN în setul de date
- Indecșii punctelor core în setul de date
- Caracteristicile punctelor core

- Dacă pentru un set de date, pe lângă X avem și y , adică output-urile, atunci există metode prin care putem vedea cât de bun este o metodă de clusterizare
- În practică, de obicei avem doar exemplarele X și evaluarea a cât de bun este o metodă de clusterizare față de alta este foarte dificilă
- Totuși, există metrici care sunt folosite în dezvoltarea de noi algoritmi de clustering
- În acest caz, se va folosi o metodă de clusterizare pe X , iar mai apoi clusterelor se vor compara cu ce există deja în y

✓
0
sec.



```
from sklearn.datasets import make_moons
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score, silhouette_score

X, y = make_moons(n_samples=1000, noise=0.1, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = KMeans(n_clusters=2, n_init=1, algorithm='lloyd', init='random', random_state=42)
clf.fit(X_train)

clusters = clf.predict(X_test)

adjusted_rand_score(y_test, clusters), normalized_mutual_info_score(y_test, clusters), silhouette_score(X_test, clusters)
```



```
(0.1325518617189177, 0.10151747829164671, 0.44537088904403654)
```

Afişarea scorului de clusterizare pe setul de test folosind 3 metode


```
✓ 0 sec.
from sklearn.datasets import make_moons
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score, silhouette_score

X, y = make_moons(n_samples=1000, noise=0.1, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = KMeans(n_clusters=2, n_init=1, algorithm='lloyd', init='random', random_state=42)
clf.fit(X_train)

clusters = clf.predict(X_test)

adjusted_rand_score(y_test, clusters), normalized_mutual_info_score(y_test, clusters), silhouette_score(X_test, clusters)

(0.1325518617189177, 0.10151747829164671, 0.44537088904403654)
```

Afişez:

- Adjusted Rand Score pe test
- Normalized Mutual Info Score pe test
- Silhouette Score pe test

Tips pentru test

Tips pentru test

- Faceți doar ce vă cere enunțul, nimic mai mult
- Dacă în enunț nu se specifică că se face split, nu trebuie să folosiți train_test_split
- Atenție la afișări

Tips pentru test

- Atenție la afișări



```
from sklearn.datasets import make_moons
from sklearn.cluster import MiniBatchKMeans
from sklearn.model_selection import train_test_split

X, y = make_moons(n_samples=1000, noise=0.1, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = MiniBatchKMeans(n_clusters=2, n_init=10, batch_size=50, init='k-means++', random_state=42)
clf.fit(X_train)

clusters = clf.predict(X_test)

clusters[10], y_test[10], clf.labels_[16], clf.cluster_centers_, -clf.score(X_test), clf.score(X_train), clf.inertia_

clf = MiniBatchKMeans(n_clusters=2, n_init=10, batch_size=50, init='random', random_state=42)
clf.fit(X_train)

clusters = clf.predict(X_test)

print("Hello")
```

Hello

- Merge să afișăm dacă punem virgulă între variabile doar dacă este ultima linie de cod din celulă

Tips pentru test

- Atenție la afișări

```
from sklearn.datasets import make_moons
from sklearn.cluster import MiniBatchKMeans
from sklearn.model_selection import train_test_split

X, y = make_moons(n_samples=1000, noise=0.1, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = MiniBatchKMeans(n_clusters=2, n_init=10, batch_size=50, init='k-means++', random_state=42)
clf.fit(X_train)

clusters = clf.predict(X_test)

clf = MiniBatchKMeans(n_clusters=2, n_init=10, batch_size=50, init='random', random_state=42)
clf.fit(X_train)

clusters = clf.predict(X_test)

print("Hello")

clusters[10], y_test[10], clf.labels_[16], clf.cluster_centers_, -clf.score(X_test), clf.score(X_train), clf.inertia_
```

```
⇒ Hello
(0,
 0,
 1,
 array([[ -0.17167842,  0.56600609],
        [ 1.25841659, -0.08337434]]),
 86.39450306674327,
-331.164093007906,
 331.164093007906)
```

- Chestia asta e de fapt un „artificiu” în Colab / notebooks. De fapt, ce se întâmplă în ultima linie este să se creeze un tuple, iar Google Colab afișează ultima valoare returnată de celulă, iar acea valoare e dată de ultima linie de cod
- Dacă rulați codul de mai sus ca un simplu fișier Python, nu se va afișa decât „Hello”

Tips pentru test

- Atenție la afișări

```
✓
0
sec.
▶ from sklearn.datasets import make_moons
  from sklearn.cluster import MiniBatchKMeans
  from sklearn.model_selection import train_test_split

X, y = make_moons(n_samples=1000, noise=0.1, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = MiniBatchKMeans(n_clusters=2, n_init=10, batch_size=50, init='k-means++', random_state=42)
clf.fit(X_train)

clusters = clf.predict(X_test)

print(clusters[10], y_test[10], clf.labels_[16], clf.cluster_centers_, -clf.score(X_test), clf.score(X_train), clf.inertia_)

clf = MiniBatchKMeans(n_clusters=2, n_init=10, batch_size=50, init='random', random_state=42)
clf.fit(X_train)

clusters = clf.predict(X_test)

print("Hello")
```

```
0 0 0 [[ 1.14988231 -0.07348405]
 [-0.28341901  0.63970077]] 88.04307483029518 -331.70820139536573 331.70820139536573
Hello
```

- Puteți afișa și cu print dacă doriți

Tips pentru test

- Ce intră la test:
 - Laboratorul 2: regresia liniară, gradient descent, regresia logistică, regresia softmax, regularized regression
 - Laboratorul 3: K-Nearest Neighbours și SVM
 - Laboratorul 4: Arbori de decizie, Voting, Bagging / Pasting, Boosting
 - Laboratorul 5: PCA, Locally Linear Embeddings, T-SNE
 - Laboratorul 6: K-Means, Agglomerative Clustering, DBSCAN, evaluarea modelelor de clustering
- Problemele vor fi similare cu cele din exerciții și ce am discutat până acum

That's it for today!
Now let's do some exercises.

That's it for today!

Now let's do some exercises.

Laboratory 6 – Exercises

1. Use the following dataset:

```
X, y = make_blobs(  
    n_samples=400,  
    centers=4,  
    cluster_std=1.25,  
    random_state=42  
)
```

in order to apply the original K-means clustering algorithm. Run the algorithm for 40 iterations and plot the model's decision boundaries.

2. The following function is used to create a dataset with 2 features:

```
def Circle(r,n):  
    return [(math.cos(2*math.pi/n*x)*r+  
            +np.random.normal(-30,30),math.sin(2*math.pi/n*x)*r+  
            +np.random.normal(-30,30)) for x in range(1,n+1)]
```

This function takes the radius and number of data points as arguments and returns an array of data points which when plotted forms a circle. In order to apply clustering algorithms, three concentric circles are created, and finally noise is added to the data as follows:

```
df=pd.DataFrame(Circle(500,1000))  
df=df.append(Circle(300,700))  
df=df.append(Circle(100,300))  
  
X=df.append([(np.random.randint(-600,600),  
              np.random.randint(-600,600)) for i in range(300)])
```

Perform the following:

- plot the dataset;
- apply the K-means++ algorithm and plot the decision boundaries;
- apply the agglomerative clustering algorithm with complete linkage and visualize the clusters;
- apply the DBSCAN algorithm with `eps=35` and `min_samples=6`, and plot the clusters.